INTEGRATED DETECTION ENGINEERING & PURPLE TEAM LABORATORY

A Fully Integrated Detection-to-Response Security Operations Environment

- FortiGate NGFW
- Suricata IDS
- Splunk SIEM
- Cowrie Honeypot
- Shuffle SOAR
- MITRE ATT&CK

Shiza

Table of Contents

Executive Summary 2
1. Network Architecture & Firewall Foundation 3
2. Intrusion Detection Engineering (Suricata) 8
3. SIEM Integration & Log Correlation (Splunk) 13
4. Structured Adversary Emulation (MITRE ATT&CK) 17
5. Deception Technology (Cowrie Honeypot) 19
6. Security Orchestration & Automated Response (Shuffle SOAR) 27
7. Detection Latency Measurement 34
8. ATT&CK Coverage Summary 34
9. Limitations and Next Steps 35
10. Repository 35

Executive Summary

This project is a self-built Security Operations Center (SOC) laboratory that covers the path from detection to response: network segmentation and firewall policy, custom intrusion detection, centralized log collection and correlation, adversary emulation, deception, and automated blocking through a firewall API.

Five platforms are connected in one environment:

- **FortiGate-VM 7.4.12** routes the attacker and victim segments, enforces policy and logs every session.
- **Suricata 8.0.3** runs as a passive sensor with custom rules for SQL injection, SSH connection bursts and SYN scanning.
- **Splunk Enterprise 10.4.2** collects FortiGate, Suricata and Cowrie data and correlates it by attacker and victim address.
- **Cowrie** is an SSH honeypot that records attacker sessions and every command typed.
- **Shuffle** is a SOAR platform whose playbook creates FortiGate address objects through the REST API.

Attacks are generated from Kali Linux, including MITRE ATT&CK-mapped tests from Atomic Red Team, and each result is checked in Suricata, FortiGate or Splunk.

What was verified

Capability	Evidence	Section
Custom detections fire on attacks that cross the firewall	Revision 2 rules re-tested from Kali through FortiGate: scan, SSH burst, URL-encoded and POST-body SQL injection; quiet on light benign traffic	2
Firewall and IDS events correlate in Splunk	126 firewall events and 1,398 IDS alerts for one attacker/victim pair, in a single search	3
Honeypot captures attacker commands with the right source	Kali session logged with source 10.10.10.10 after a firewall bypass was found and fixed	5
Automated block works	Shuffle creates the address object, a deny policy blocks traffic: ping loss 0% before, 100% after, deny entries in the firewall log	6
Playbook is idempotent	Second run finds the object (HTTP 200) and skips creation	6

What is partial. The playbook was triggered manually with a test payload, not by a live honeypot event, and adding a new object to the deny policy's address group is a manual step. Section 9 lists these and other limits, together with the next steps.

Troubleshooting is documented where it changed the design: a routing path that bypassed the firewall, a honeypot log that reached Splunk as one merged event, a SOAR database that crashed under memory pressure, and a block object that was created but not enforced by any policy.

Screenshots come from the working lab.

1. Network Architecture & Firewall Foundation

The lab uses an Attacker segment (`10.10.10.0/24`, FortiGate `port2`) and a Victim segment (`10.10.20.0/24`, FortiGate `port3`). A separate management network (`10.10.30.0/24`) holds Splunk and Shuffle outside the attacker–victim path. FortiGate sends syslog to Splunk.

Kali and the Cowrie VM each have an extra adapter on VMware's NAT network (`192.168.75.0/24`). That created a side path around the firewall, which was found and fixed during testing (Section 5).

Segment	Network	Hosts
Attacker	10.10.10.0/24 (port2)	Kali (10.10.10.10)
Victim	10.10.20.0/24 (port3)	Metasploitable2 (10.10.20.10), Cowrie (10.10.20.50), Suricata sensor
Management	10.10.30.0/24 (VMnet4)	Splunk (10.10.30.10), Shuffle (10.10.30.20)
NAT / WAN	192.168.75.0/24 (port1)	Package installs and admin access; also the side path

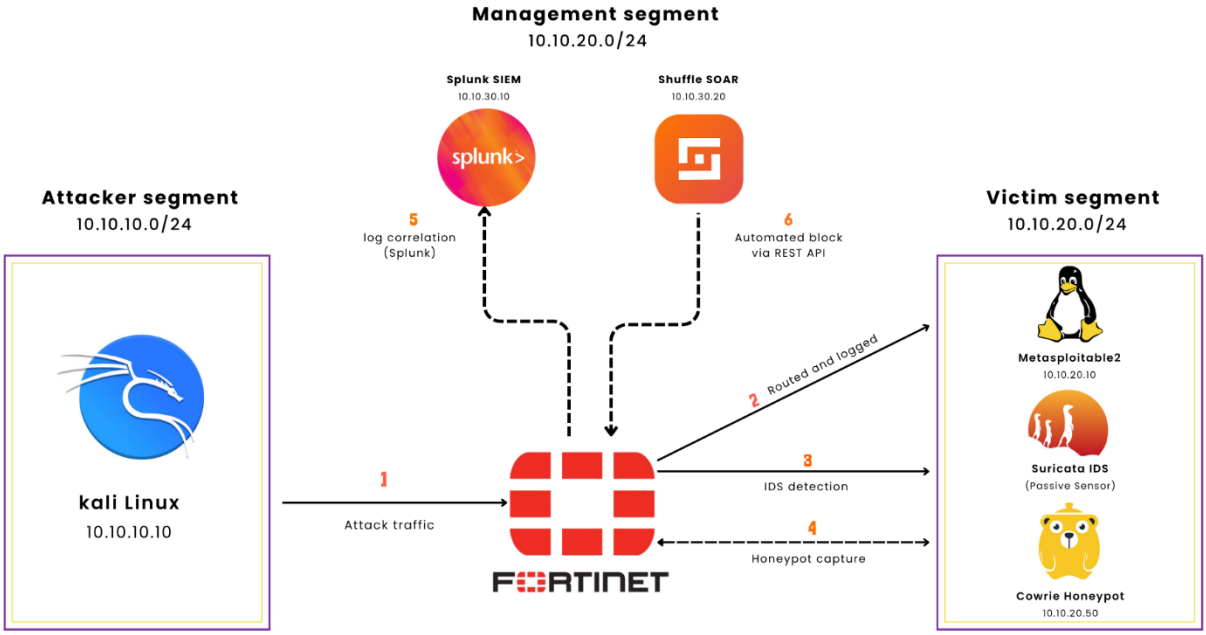


Figure 1.1 — Lab topology, drawn from the interface table, host configuration and VMware adapter addresses captured during the build.

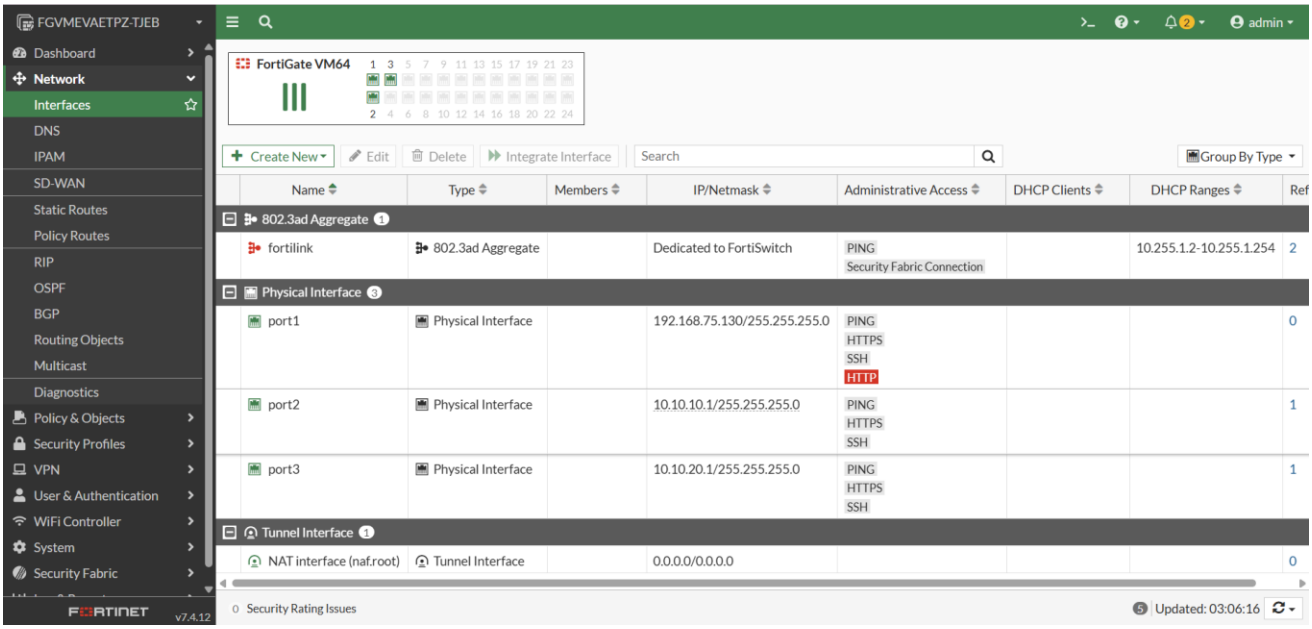


Figure 1.2 — FortiGate interface table: port2 (10.10.10.1) and port3 (10.10.20.1) carry the lab segments; port1 sits on the VMware NAT network.

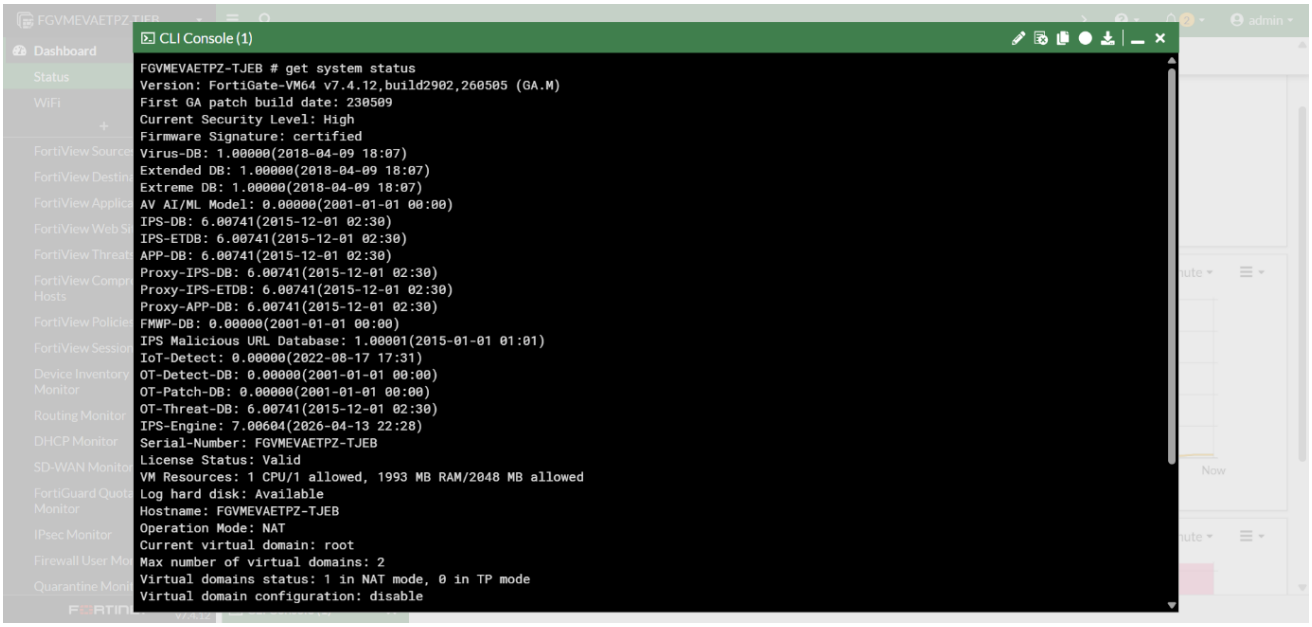


Figure 1.3 — FortiOS 7.4.12 confirmed with get system status. The VM runs on an evaluation license, and the signature databases carry 2015–2018 dates.

Because the evaluation license has no signature updates and the policy uses no inspection profile, FortiGate acts as the routing, policy and logging point. Suricata performs the detection.

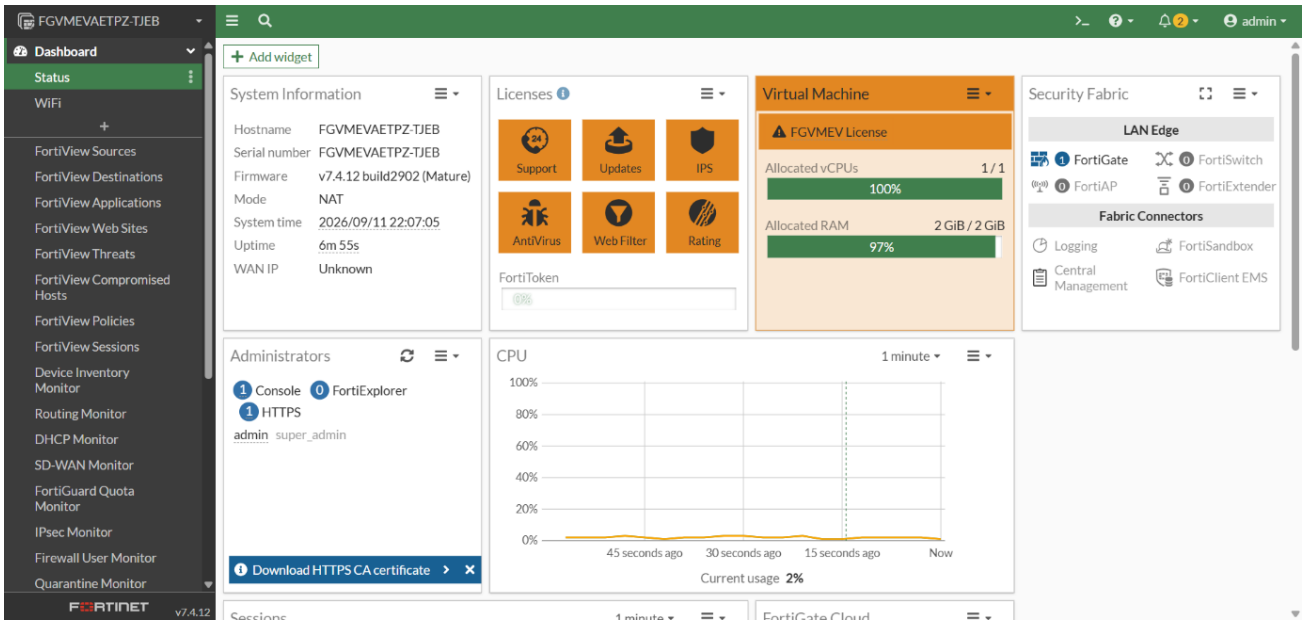


Figure 1.4 — FortiGate dashboard: system information, license status and resource utilization.

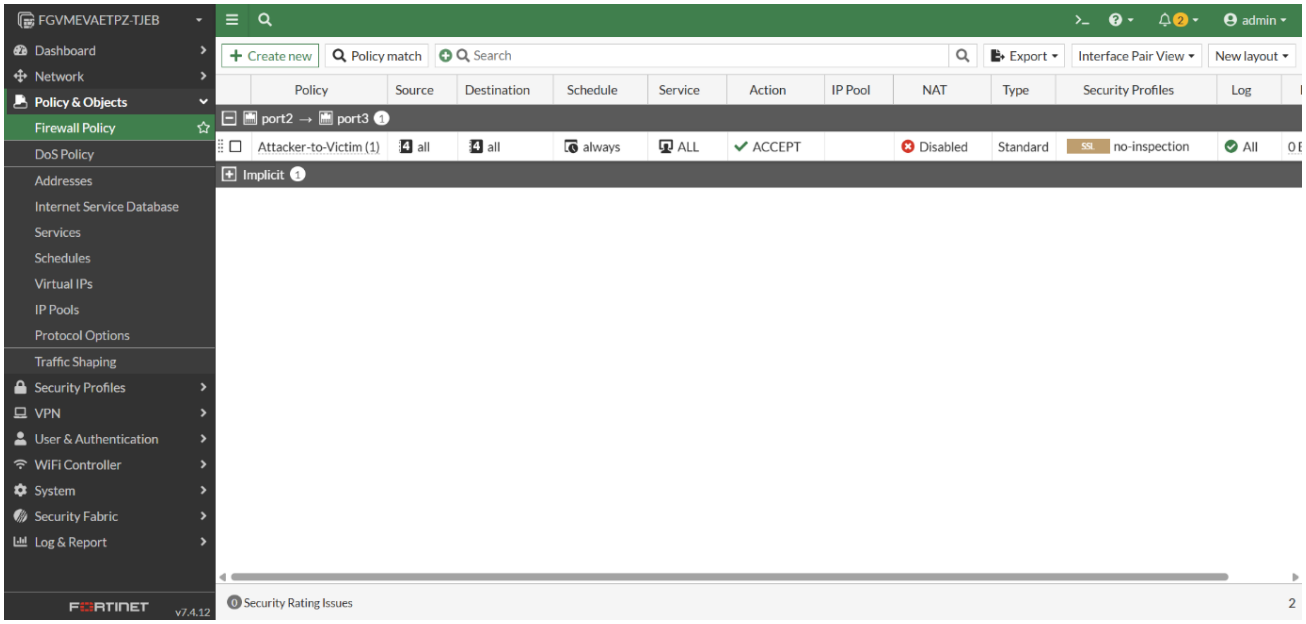


Figure 1.5 — The original Attacker-to-Victim policy: all services accepted and logged, no inspection. A deny policy was added later (Section 6).

End-to-end connectivity through the firewall was verified from Kali.

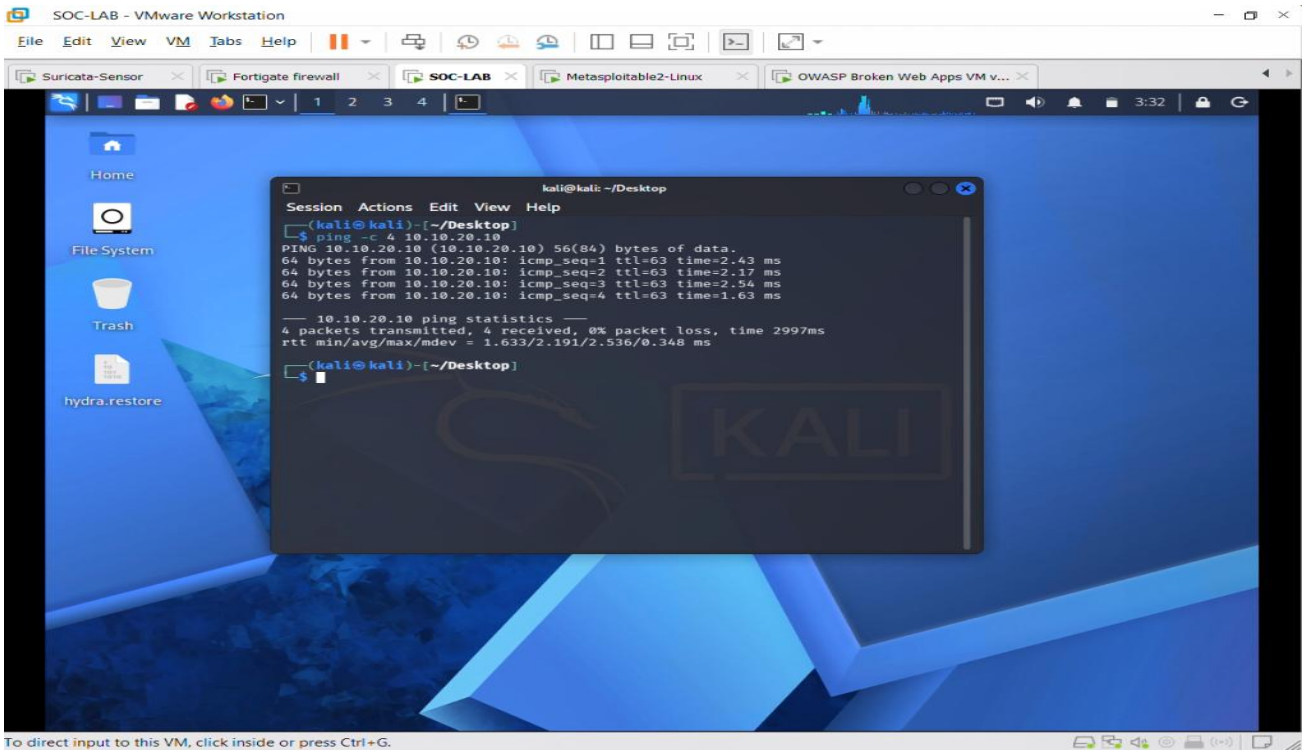


Figure 1.6 — Kali pinging the victim host through FortiGate, 0% packet loss.

Every session that crosses the firewall appears in the Forward Traffic log, which is the firewall's own record and later a Splunk data source.

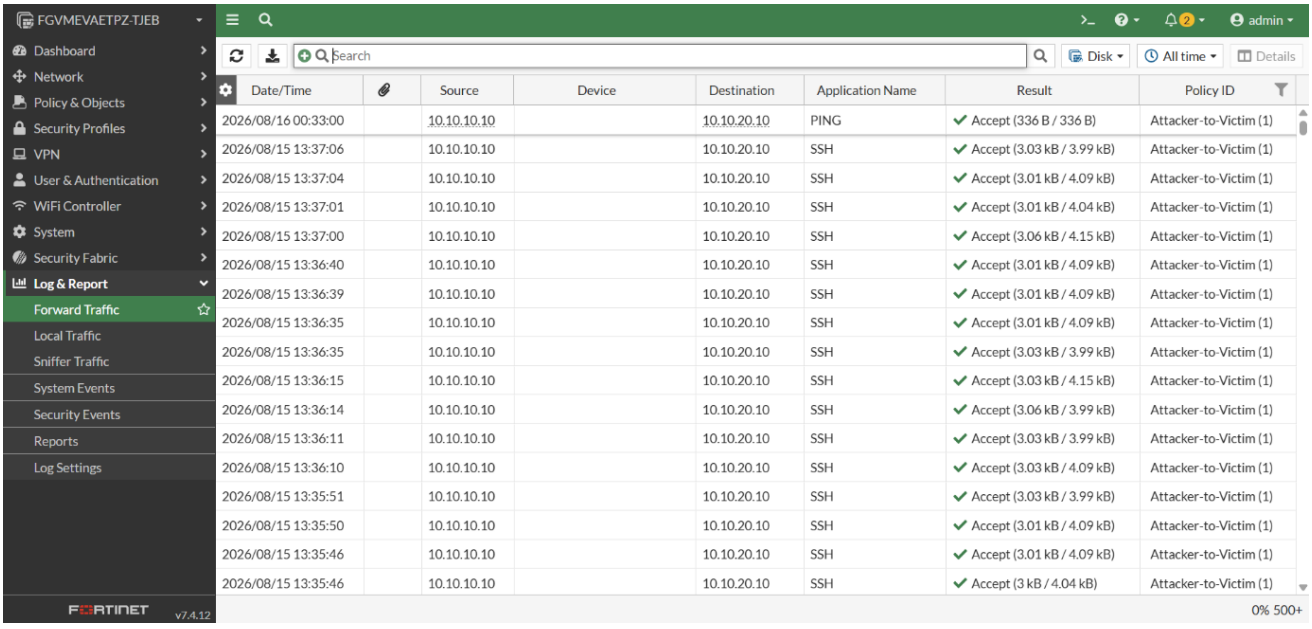


Figure 1.7 — Forward Traffic log: policy-matched SSH sessions during brute-force testing.

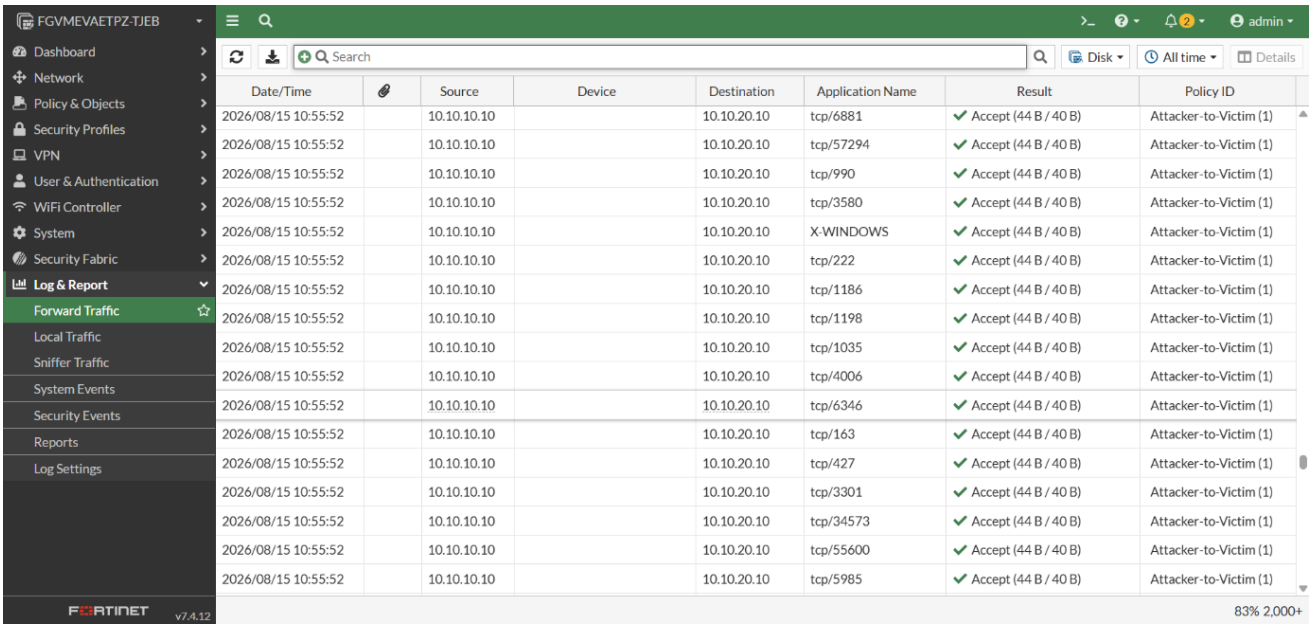


Figure 1.8 — Forward Traffic log during a port scan: many short probe sessions, each accepted and logged.

2. Intrusion Detection Engineering (Suricata)

Suricata runs as a passive sensor in promiscuous mode on the Victim segment, giving visibility independent of the firewall's own logging. Three custom rules cover three techniques: UNION-based SQL injection, SSH connection bursts and SYN scanning.

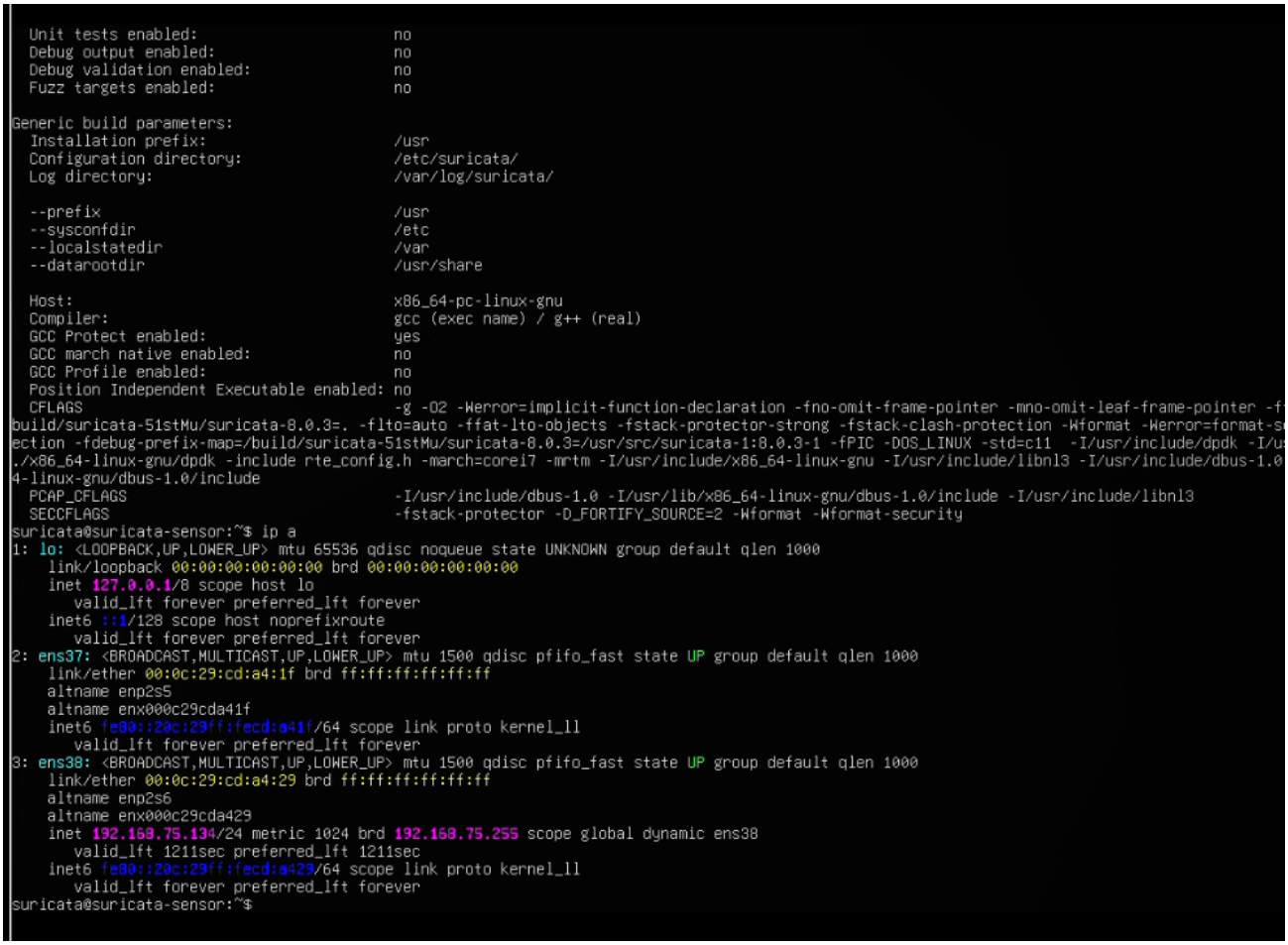


Figure 2.1 — Suricata 8.0.3 built and running on the sensor, with the monitoring interface bound.

Revision 1: first detections

Each rule was first tested against self-generated attack traffic and confirmed to fire, with the alerts visible in Splunk.

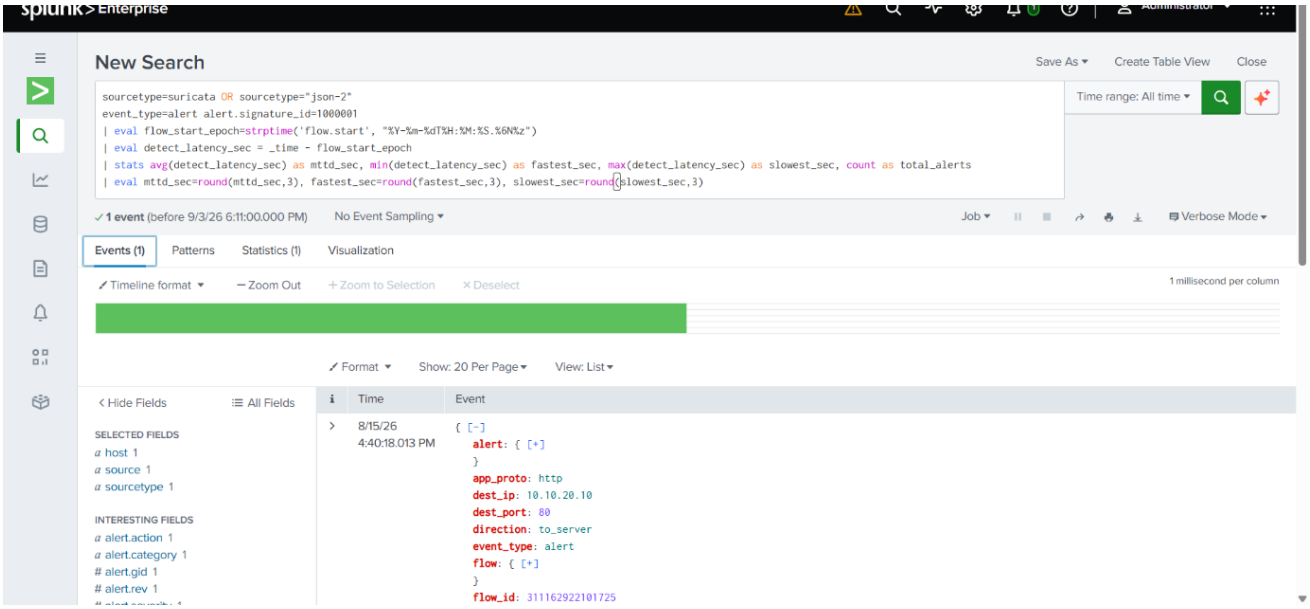


Figure 2.2 — SQL injection rule (SID 1000001, revision 1) firing on a UNION SELECT payload, visible in Splunk.

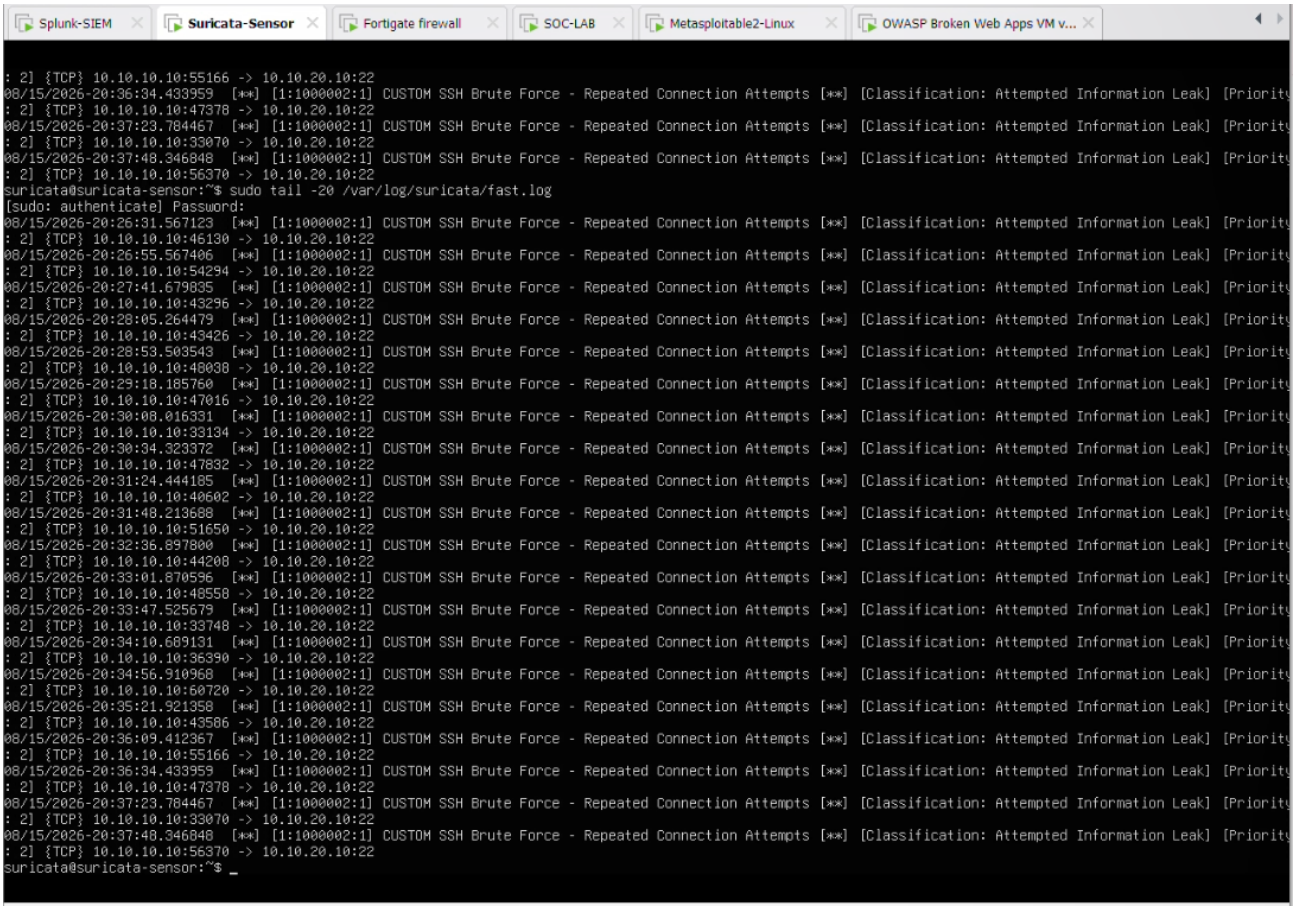


Figure 2.3 — SSH rule (SID 1000002, revision 1) alerting on repeated connections to port 22.

Revision 1 had three weaknesses found in review: the SQL injection rule matched raw payload bytes, so encoding could evade it; the threshold rules alerted on every packet after the count was reached (one Hydra run produced 1,079 alerts); and the messages overstated what the rules measure, since they count TCP SYN packets, not authentication attempts or distinct ports.

Revision 2: tuned rules

SID	Change in revision 2
1000001	Matches the decoded HTTP URI using a sticky buffer, so URL-encoded keywords are caught
1000004 (new)	Same keyword match on the HTTP request body
1000002	Threshold type `both`: alerts once per interval; message renamed to connection attempts
1000003	Threshold type `both`; message renamed to high-rate SYN scan

Revision 2 rules (also in the repository)

```
alert http any any -> $HOME_NET any (msg:"CUSTOM SQLi - UNION SELECT in URI";
  flow:established,to_server; http.uri; content:"union"; nocase;
  content:"select"; nocase; distance:0; classtype:web-application-attack;
  sid:1000001; rev:2;)
alert http any any -> $HOME_NET any (msg:"CUSTOM SQLi - UNION SELECT in request
body";
  flow:established,to_server; http.request_body; content:"union"; nocase;
  content:"select"; nocase; distance:0; classtype:web-application-attack;
  sid:1000004; rev:1;)
alert tcp any any -> $HOME_NET 22 (msg:"CUSTOM SSH - Repeated Connection Attempts";
  flow:to_server; flags:S; threshold:type both, track by_src, count 5, seconds 60;
  classtype:attempted-recon; sid:1000002; rev:2;)
alert tcp any any -> $HOME_NET any (msg:"CUSTOM Network - High-Rate TCP SYN Scan";
flags:S;
  threshold:type both, track by_src, count 15, seconds 10;
  classtype:network-scan; sid:1000003; rev:2;)
```

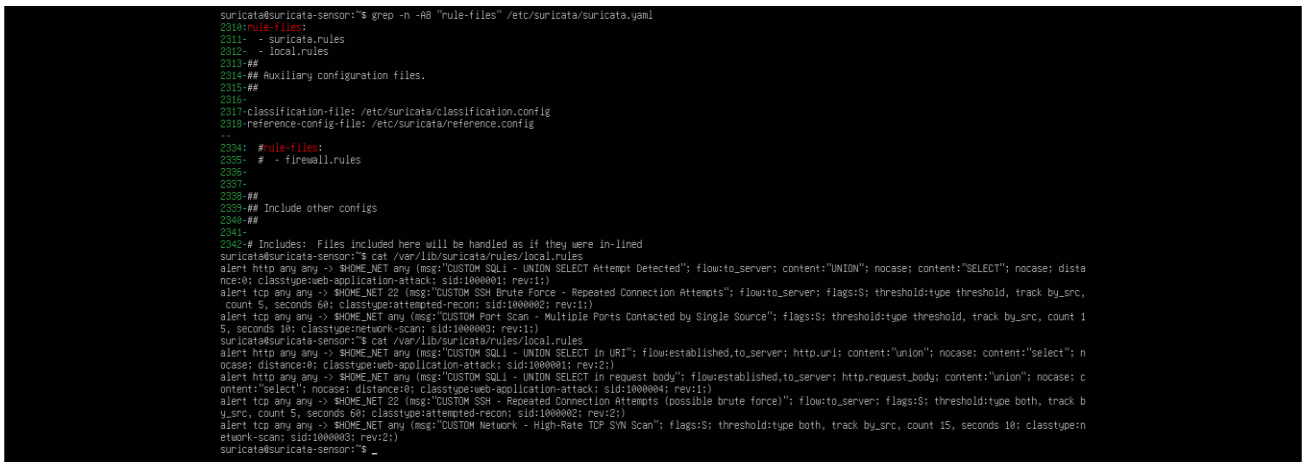


Figure 2.4 — The live local.rules file on the sensor: the revision 1 rules (first listing) and the revision 2 rules that replaced them (second listing).

The rules were validated with Suricata's test mode before the live swap, then loaded and tested from Kali through the firewall.

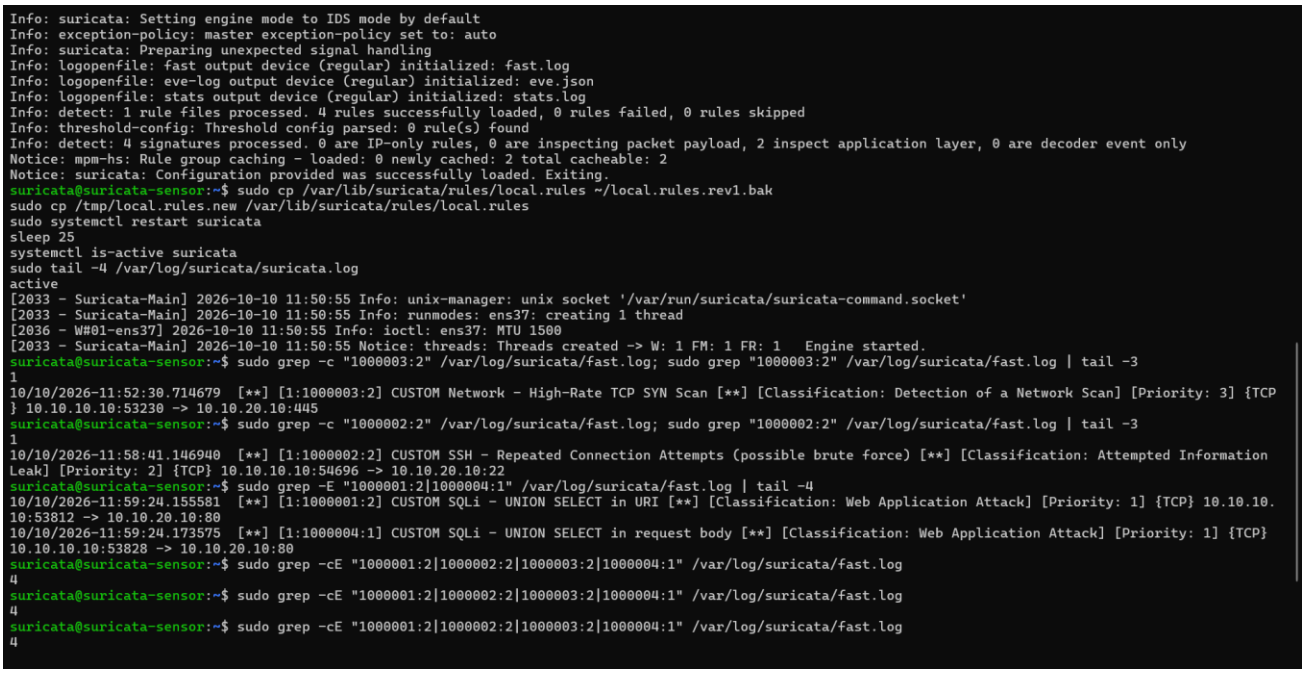


Figure 2.5 — Rule validation with suricata -T (4 rules loaded, 0 failed), restart of the engine, one alert per rule from the re-test, and the alert count staying at 4 after benign traffic.

Test (from Kali, through FortiGate)	Result
`nmap -sS -p 1-1000` against the victim	1 alert, SID 1000003 (revision 1 produced hundreds)
10 connections to port 22	1 alert, SID 1000002
URL-encoded `%55NION ALL %53ELECT` in the URI	Alert, SID 1000001
`UNION SELECT` in a POST body	Alert, SID 1000004
Benign traffic: 2 HTTP requests, 3 pings, nmap of 3 ports	No new alerts



Figure 2.6 — Test traffic from Kali through FortiGate: an Nmap SYN scan, a connection burst to port 22, and SQL injection payloads in the URL and in a POST body. A Hydra attempt failed on SSH algorithm negotiation, so the burst was used instead.



Figure 2.7 — Benign traffic from Kali (HTTP requests, pings, a three-port Nmap scan), which raised no new alerts.

The SSH test used a connection burst because Hydra on current Kali cannot negotiate the legacy MAC algorithms offered by Metasploitable's SSH server, and the rule counts connections in any case. The benign-traffic check was small and from one source; it is not a false-positive study.

3. SIEM Integration & Log Correlation (Splunk)

Splunk Enterprise is the central collection point. FortiGate sends syslog to Splunk, and a Universal Forwarder on the Suricata sensor ships `eve.json`. Deployment surfaced two infrastructure problems that were diagnosed rather than worked around: a non-interactive password-seeding failure, resolved with Splunk's `--seed-passwd` start flag, and a port-514 conflict with the host's own logging service.

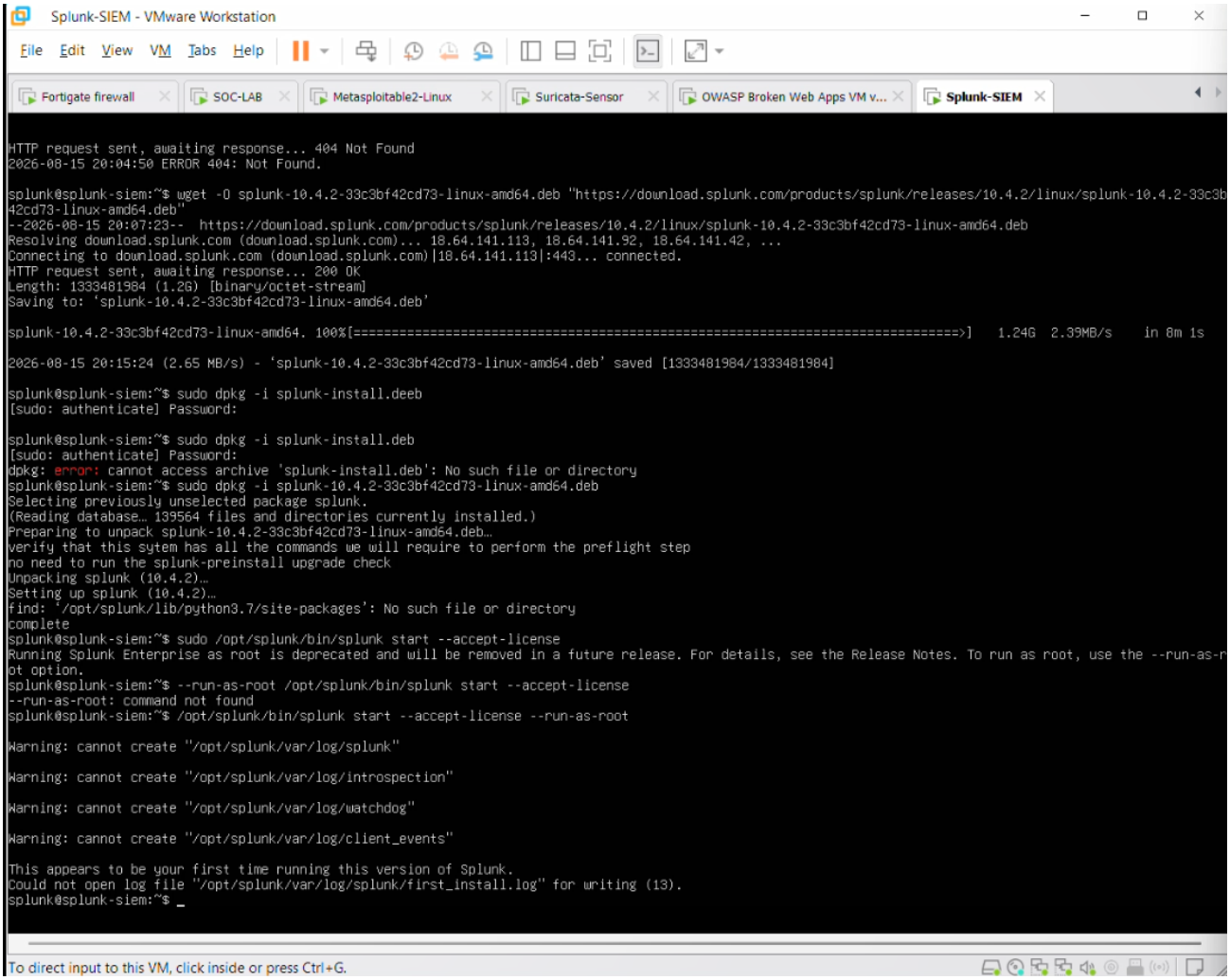


Figure 3.1 — Diagnosing a Splunk startup failure: the deprecated root-execution warning and the start flag that resolved it.

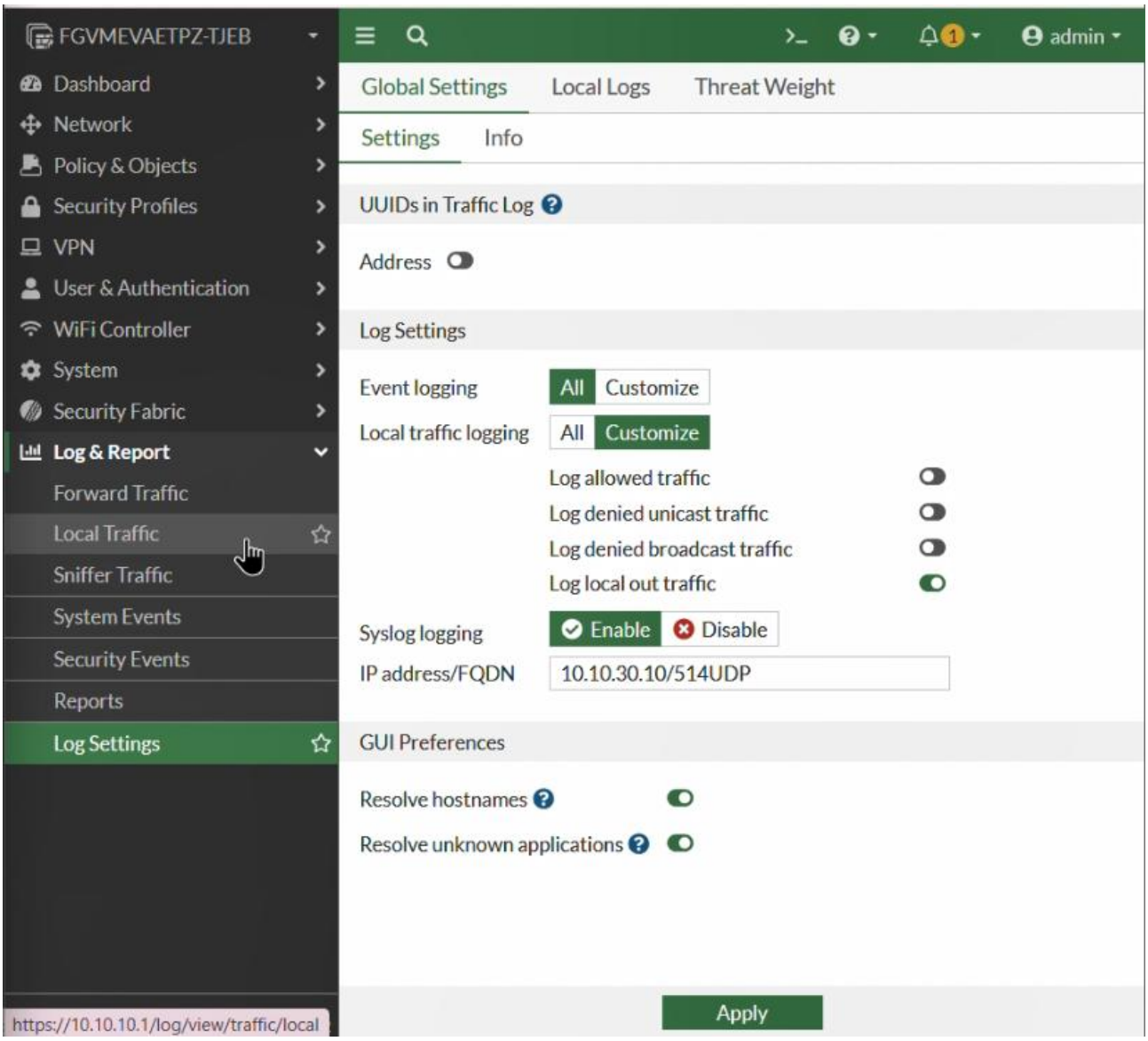


Figure 3.2 — FortiGate log settings: syslog enabled and pointed at the Splunk server (10.10.30.10).

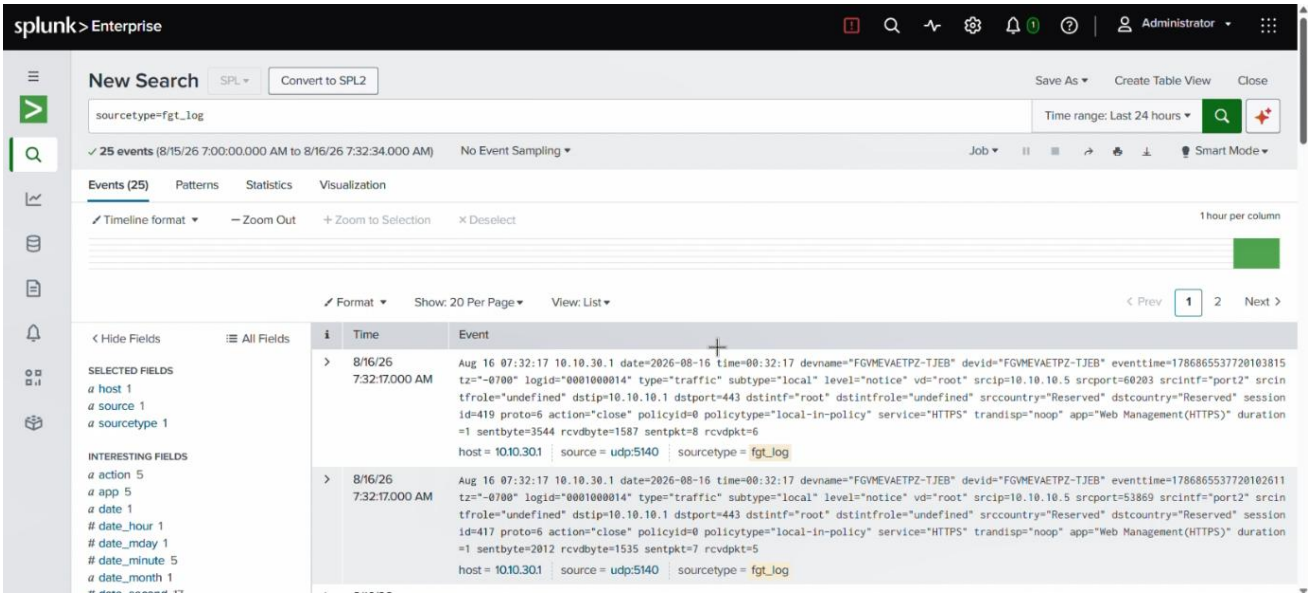


Figure 3.3 — FortiGate syslog events in Splunk (sourcetype fgt_log). These examples are management-plane sessions; forward-traffic events are used in the correlation below.

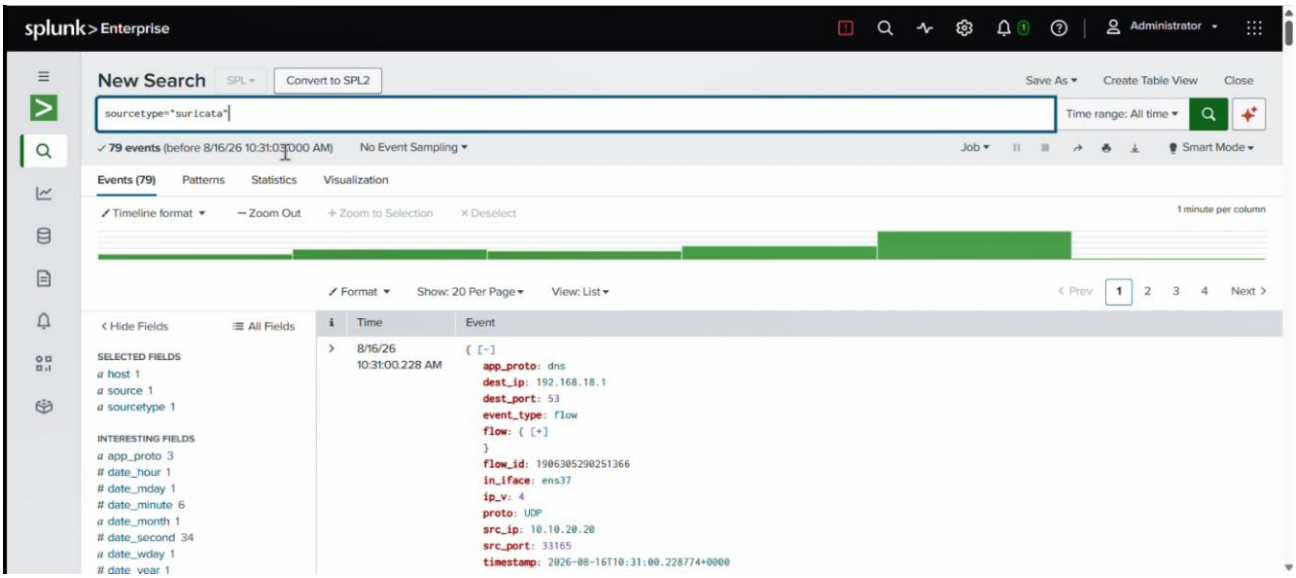


Figure 3.4 — Suricata events arriving in Splunk from the Universal Forwarder.

Cross-source correlation

One search combines the FortiGate and Suricata sources by attacker and victim address, showing how many firewall events and IDS alerts exist for each pair, which signatures fired, and which firewall policy and actions applied.

Correlation search (repository: splunk-searches/correlate-firewall-ids-by-attacker.spl)

```
(sourcetype=fgt_log OR sourcetype=suricata OR sourcetype="json-2")
| eval src=coalesce(srcip, src_ip), dst=coalesce(dstip, dest_ip)
| where src="10.10.10.10"
| stats count(eval(sourcetype="fgt_log")) as firewall_events,
        count(eval(event_type="alert")) as ids_alerts,
        values(alert.signature) as ids_signature,
        values(policyname) as fw_policy, values(action) as fw_action by src, dst
| where firewall_events>0 OR ids_alerts>0
```

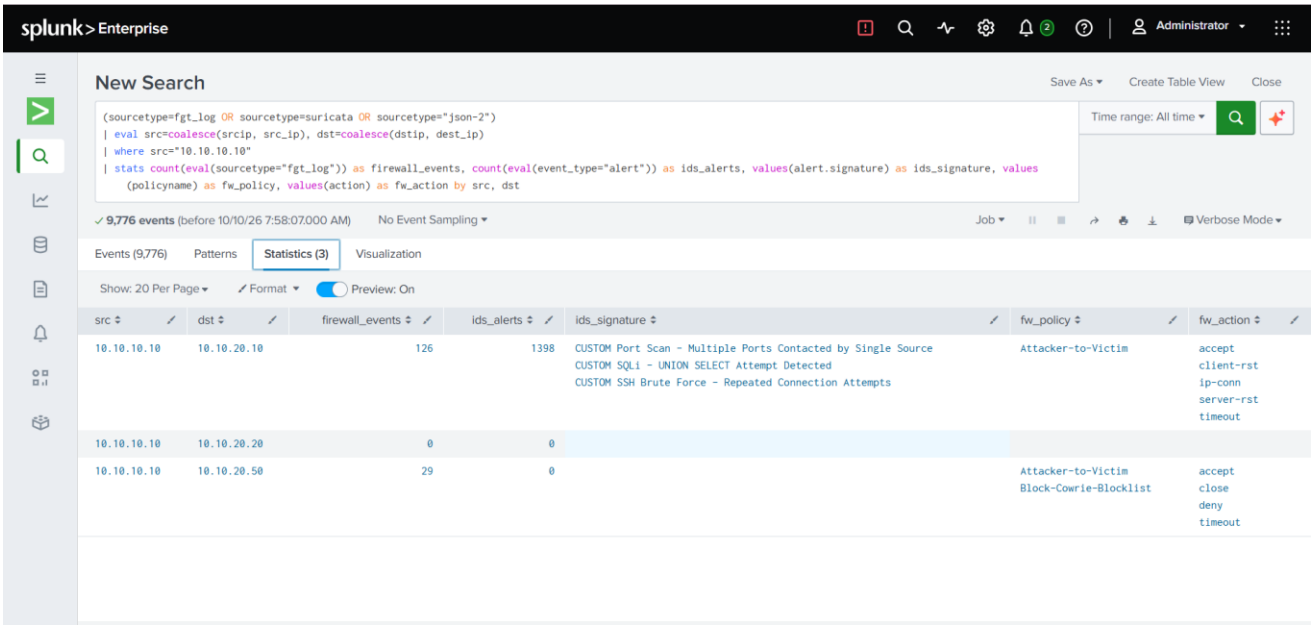


Figure 3.5 — Correlation result: the same attacker seen by both the firewall and the IDS.

Source	Destination	Firewall events	IDS alerts	Notes
10.10.10.10	10.10.20.10	126	1,398	All three custom signatures; policy Attacker-to-Victim
10.10.10.10	10.10.20.50	29	0	Includes deny actions under Block-Cowrie-Blocklist

The counts are cumulative over all testing. The join is on address, not time, so it shows the same attacker seen by two sources, not event ordering. Forwarding applies from the time syslog was enabled, so earlier sessions exist only in FortiGate's local log.

4. Structured Adversary Emulation (MITRE ATT&CK)

Where possible, detections were exercised with Atomic Red Team: numbered tests mapped to ATT&CK technique IDs, which makes the evidence reproducible. Where the library has no applicable test, the gap was checked directly rather than assumed.

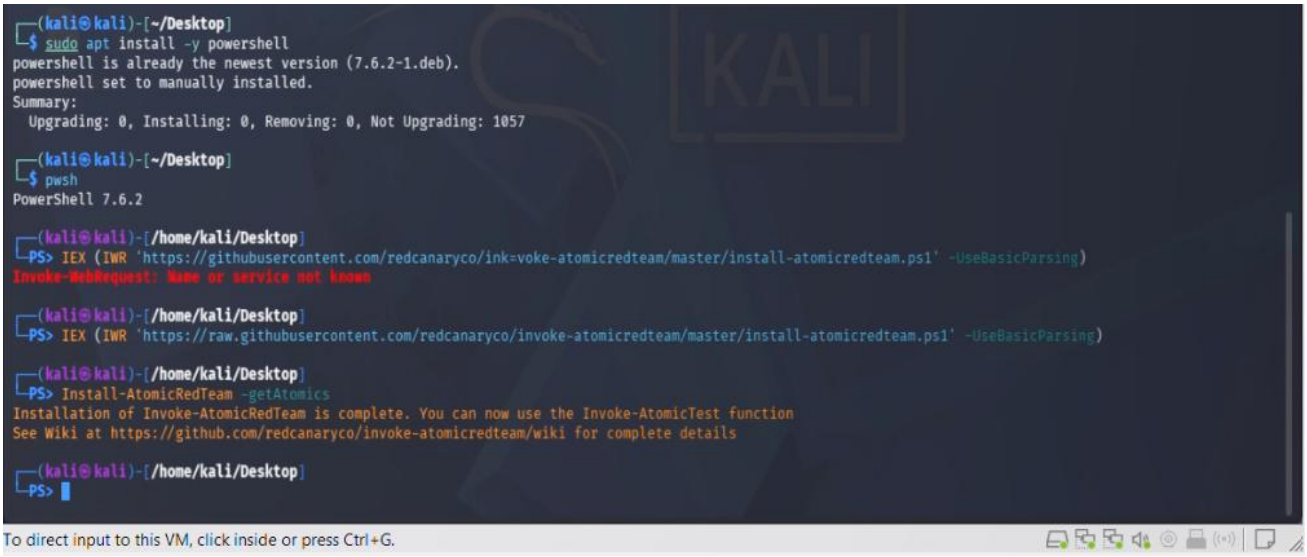


Figure 4.1 — Atomic Red Team installed on the attacker host, ready to run technique-mapped tests.

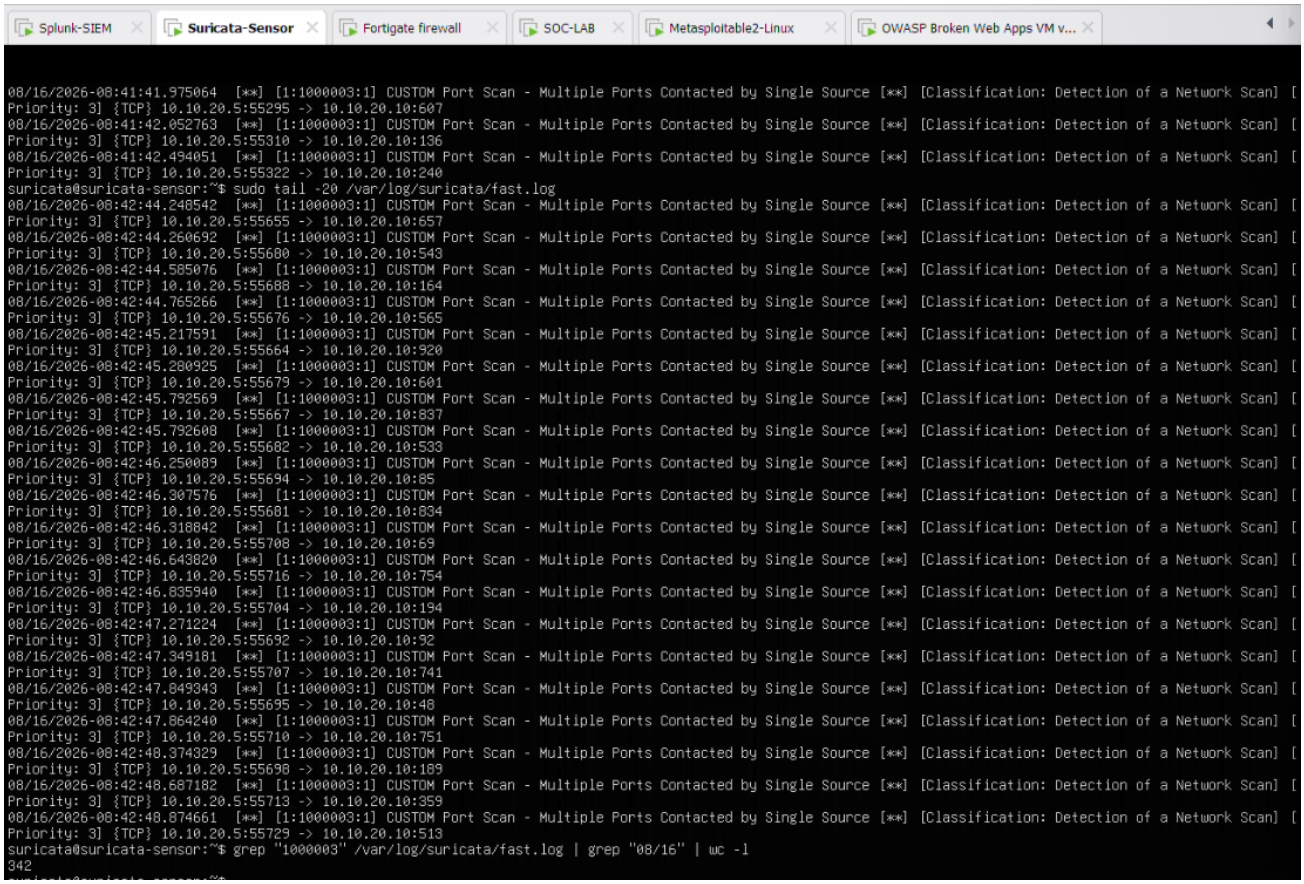


Figure 4.2 — Atomic test T1046-12 (Network Service Discovery): 342 revision-1 SYN-scan alerts counted in Suricata's alert log.

Path note. In the Suricata alerts for this run the source address is `10.10.20.5`, not Kali's `10.10.10.10`. That run reached the victim through the NAT side path (Section 5), so Suricata saw the scan but FortiGate did not log it. The later revision-2 tests (Section 2) crossed the firewall and appear in its logs.

Coverage gaps, checked directly

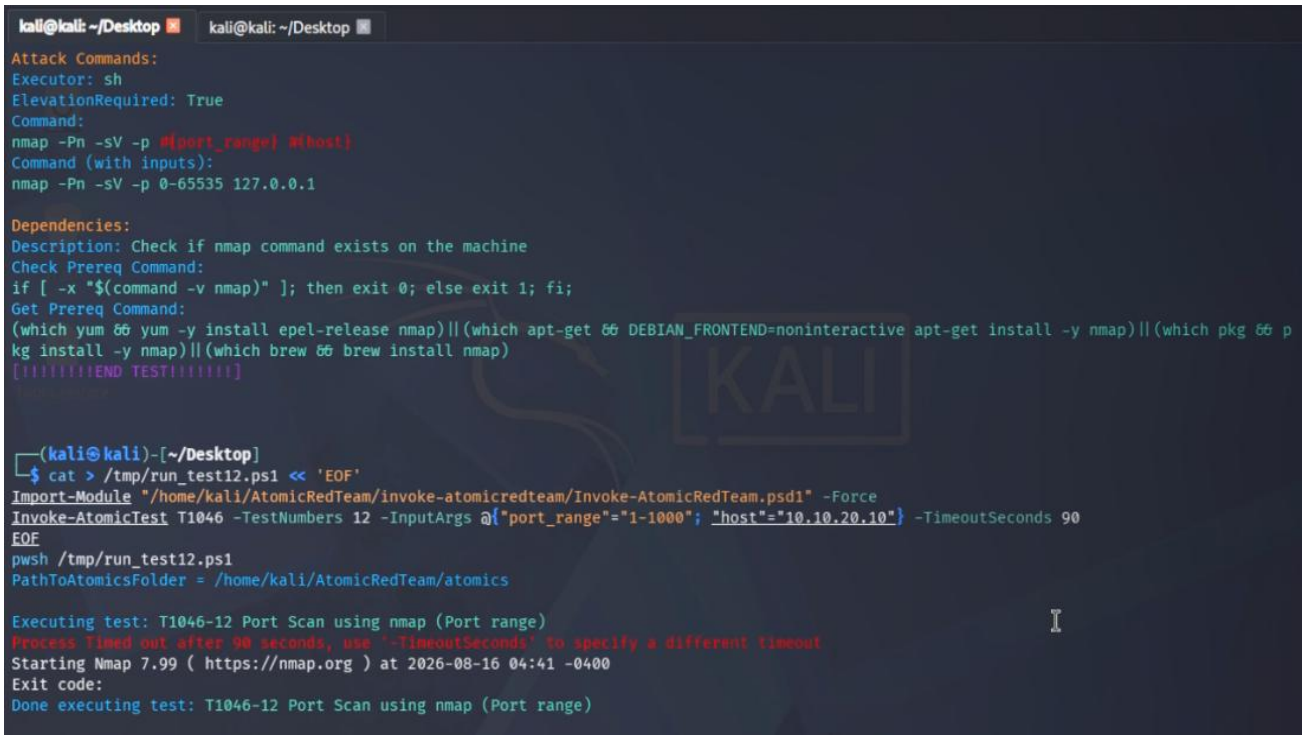


Figure 4.3 — Atomic test T1046-12 (port scan with Nmap) executed from Kali against the victim host, ports 1 to 1000.



Figure 4.4 — Atomic Red Team has no test for T1190: T1190.yaml does not exist.

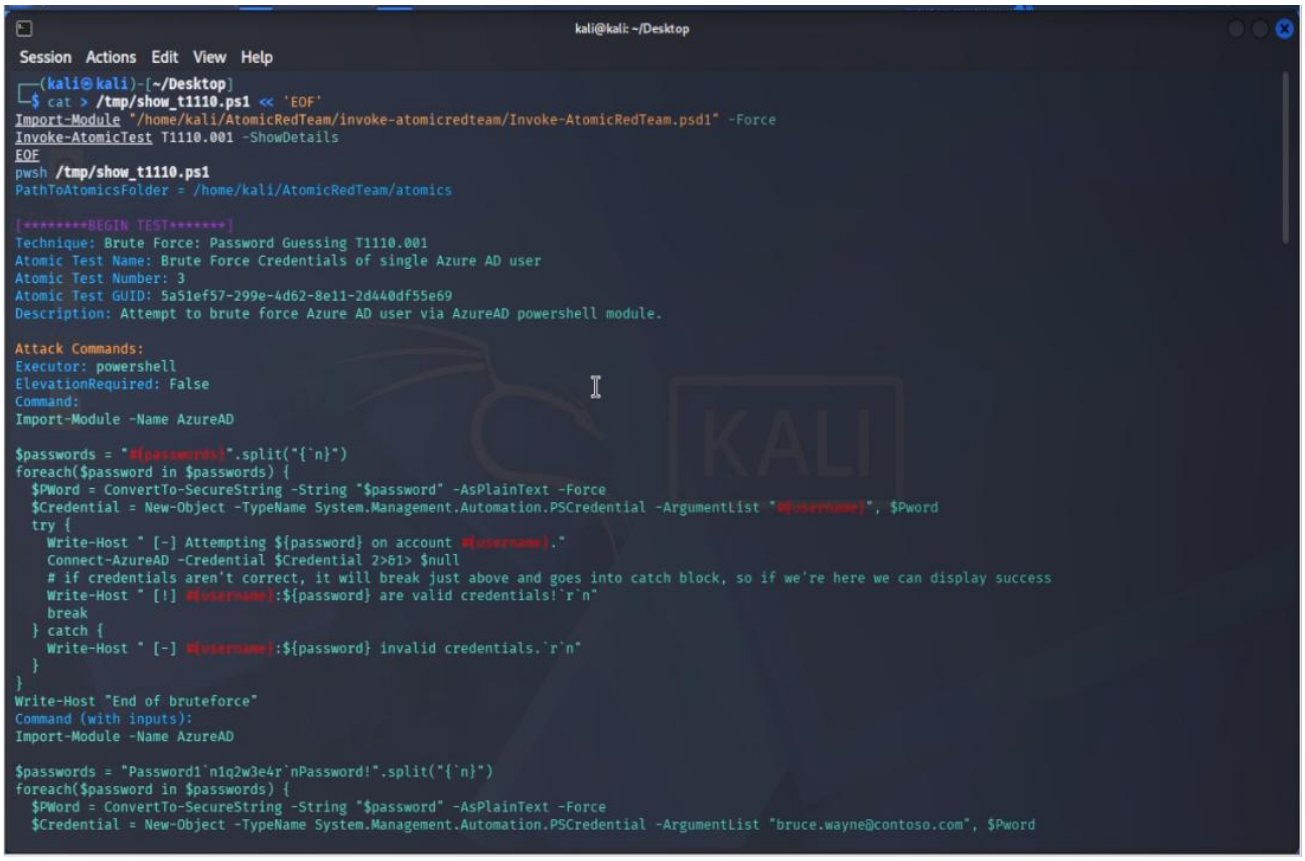


Figure 4.5 — T1110.001 test 3 is an Azure AD password-guessing test, not a network SSH brute force.

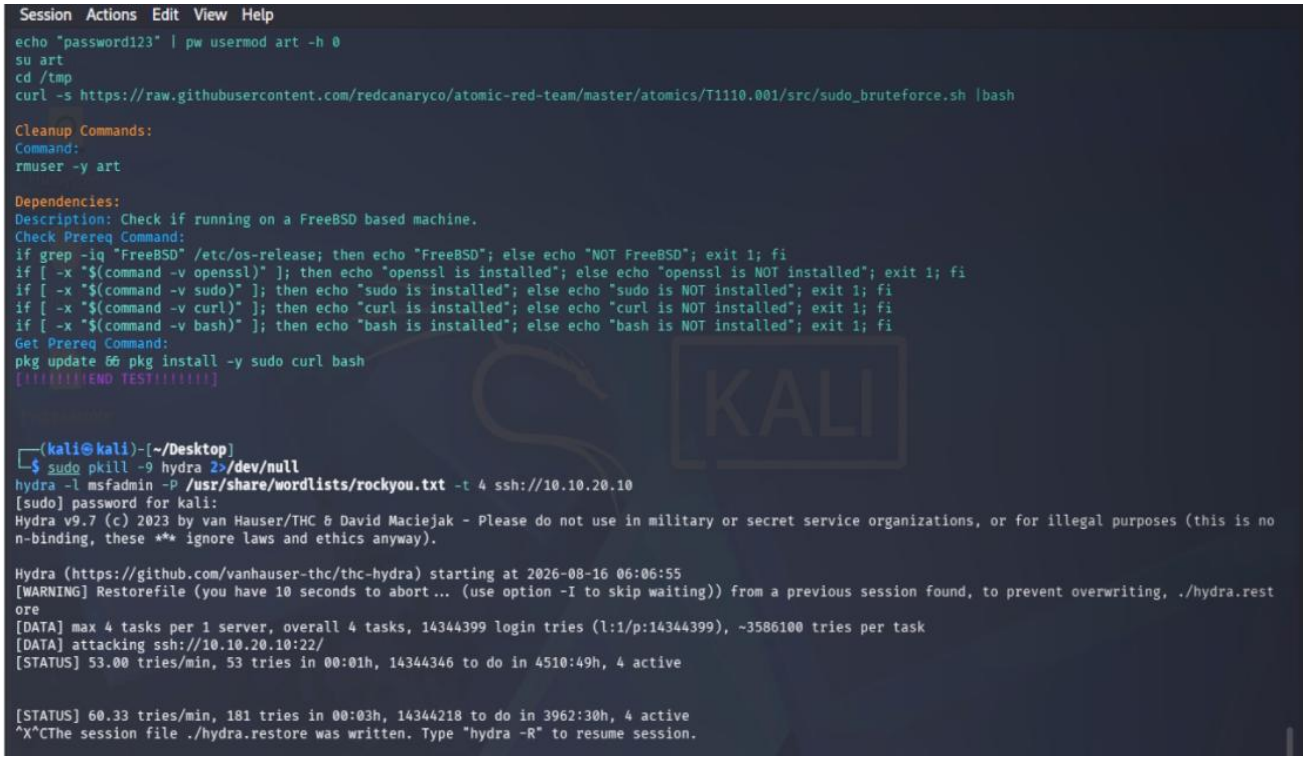


Figure 4.6 — T1110.001 also has local sudo brute-force tests (this one is the FreeBSD variant); none targets network SSH.

5. Deception Technology (Cowrie Honeypot)

A Cowrie SSH honeypot runs on the Victim segment as a high-confidence detection layer: no legitimate service has a reason to connect to it. Deployment required resolving a Python packaging issue (the

documented `bin/cowrie` entry point did not exist in the installed version) and a port-redirect misconfiguration, both diagnosed from process and network inspection.

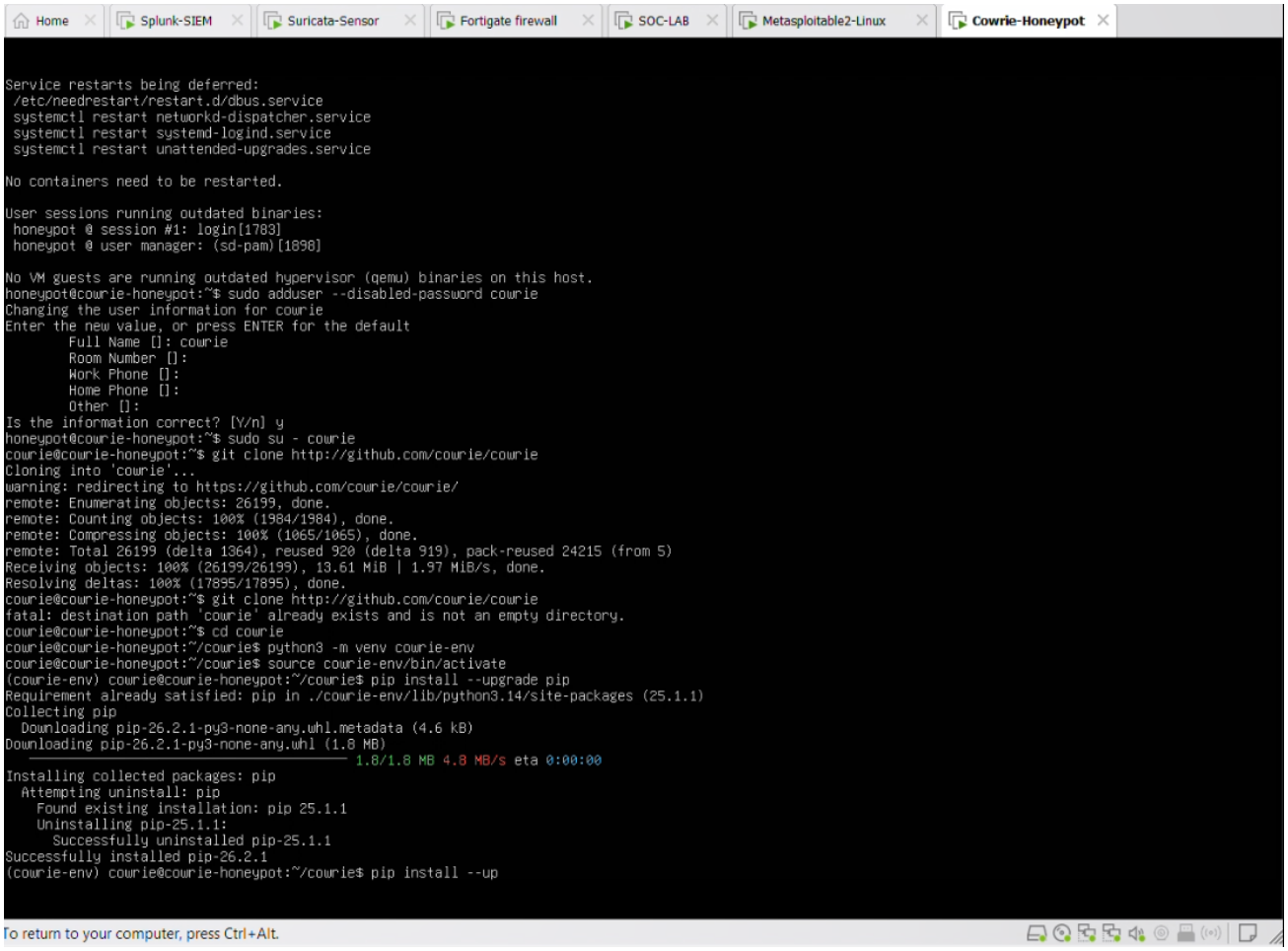


Figure 5.1 — Cowrie installed under a dedicated non-privileged service account.

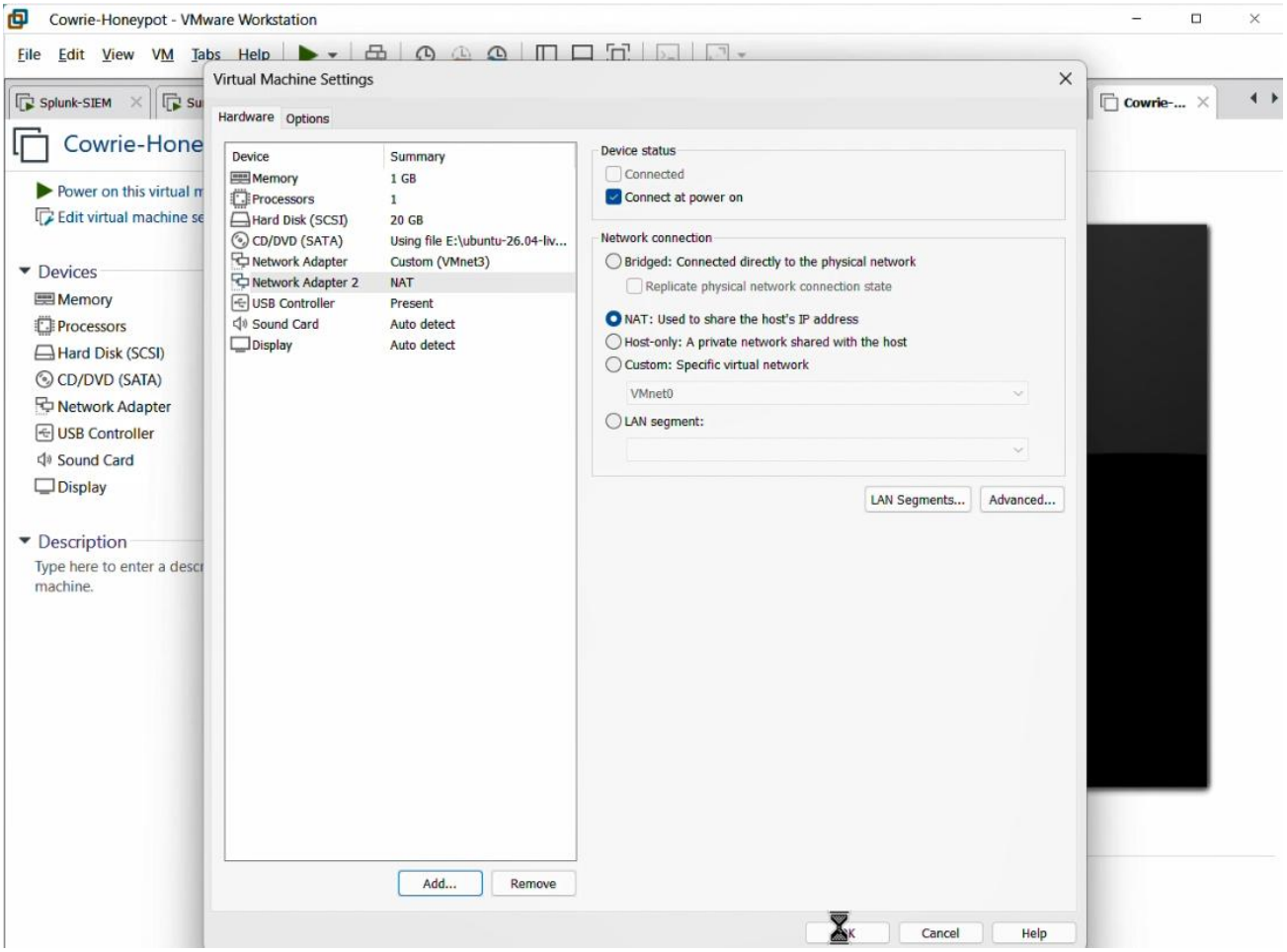


Figure 5.2 — Cowrie VM network settings. A second adapter on the NAT network later mattered for the routing finding.

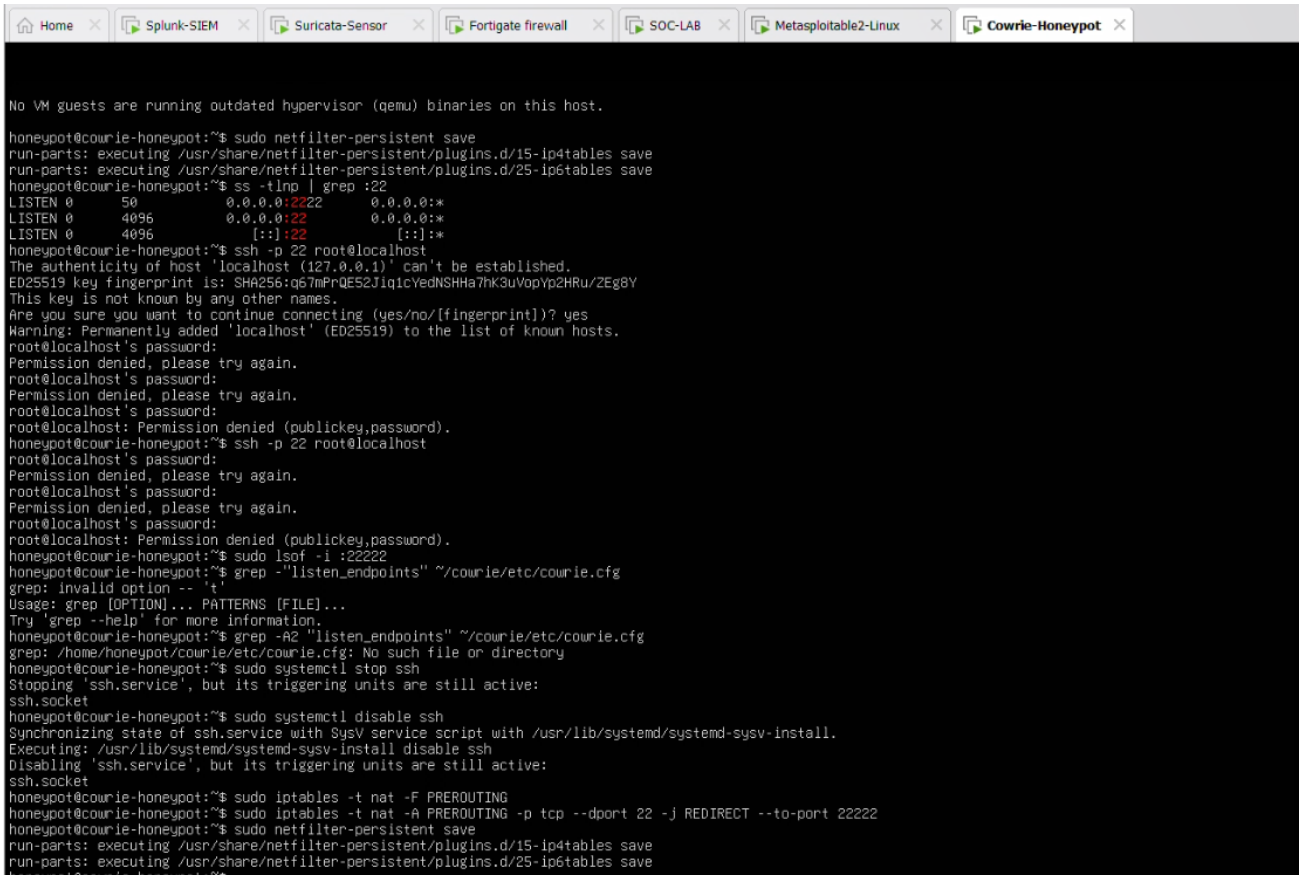


Figure 5.3 — Correcting the iptables redirect so external port-22 traffic reaches Cowrie's listener.

Finding: the first Kali test bypassed the firewall

Cowrie's JSON log recorded the commands typed from Kali, but with source `10.10.20.5` instead of Kali's `10.10.10.10`. The earlier session at 11:50Z is a local self-test from `127.0.0.1`.

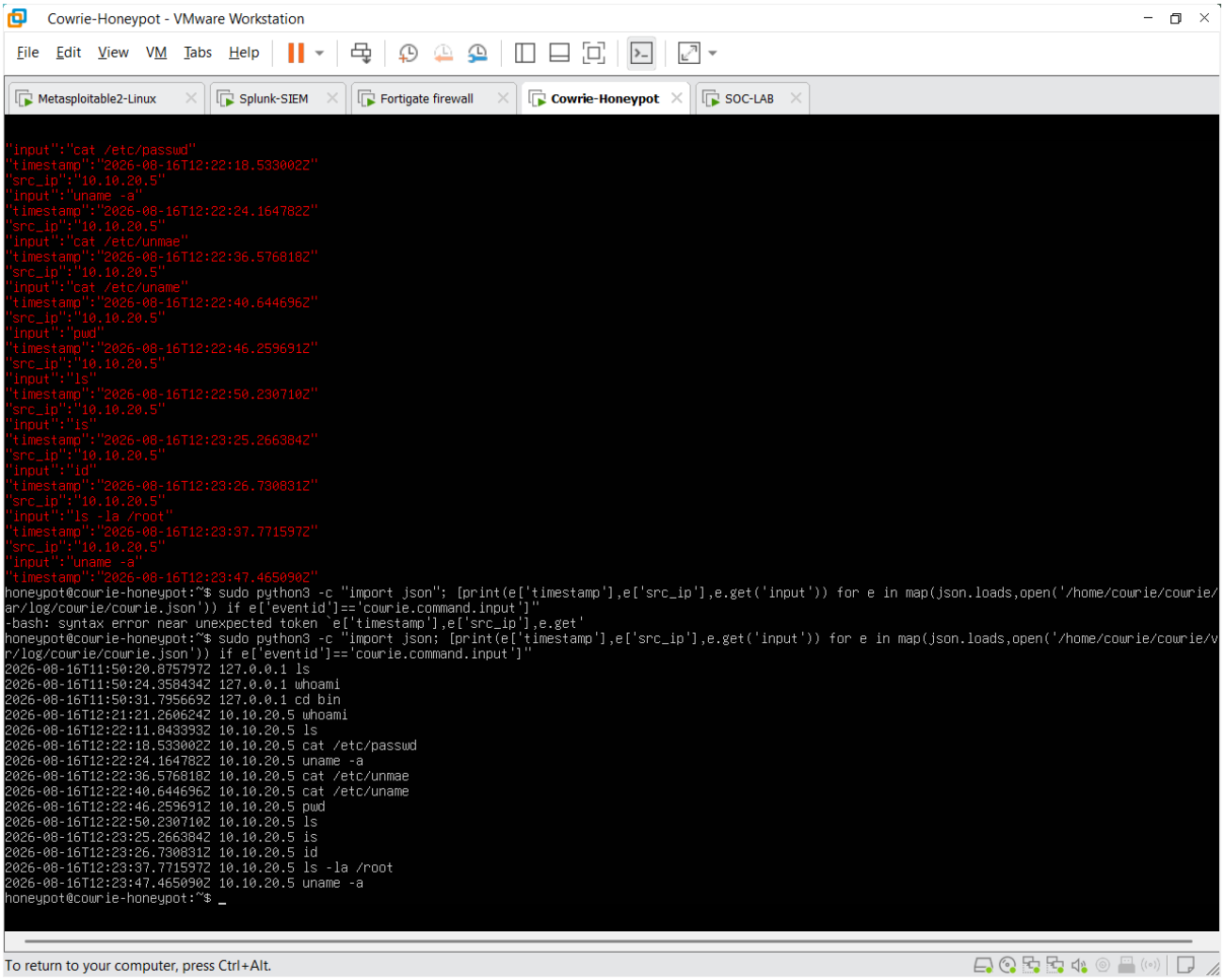


Figure 5.4 — Cowrie's log of command inputs: a local test (127.0.0.1) and the Kali session (12:21Z onward) recorded as 10.10.20.5.

The cause was found by checking the path on each side. `10.10.20.5` turned out to be the Windows host's own VMnet3 adapter. Kali's route to the victim segment pointed at the VMware NAT gateway, not the firewall, and the Cowrie VM's default route also pointed at the NAT side.

Path evidence

```
$ ip route get 10.10.20.50          (on Kali)
10.10.20.50 via 192.168.75.2 dev eth1 src 192.168.75.136

> ipconfig | findstr /i "adapter IPv4"   (on the Windows host, excerpt)
VMware Network Adapter VMnet2:   10.10.10.5
VMware Network Adapter VMnet3:   10.10.20.5
VMware Network Adapter VMnet4:   10.10.30.1
```

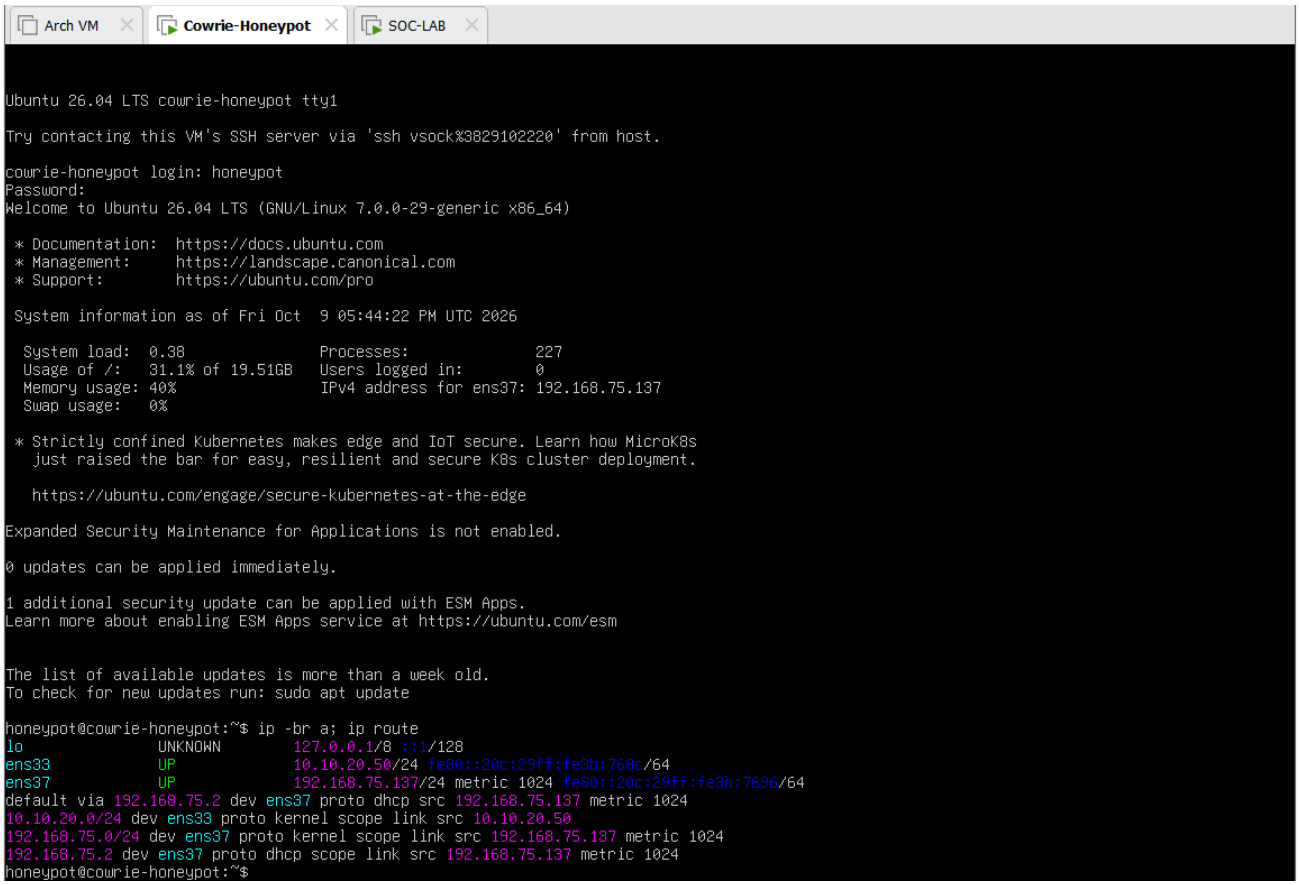


Figure 5.5 — Cowrie VM: ens33 on the victim segment, ens37 on NAT, default route via the NAT gateway.

Fix and re-test through the firewall

Routes were pinned on both ends so traffic and its replies cross FortiGate. Without the return route on the honeypot, the connection timed out because replies left through the NAT interface.

Routing fix and verification

```
# Kali
sudo ip route replace 10.10.20.0/24 via 10.10.10.1 dev eth0
# Cowrie VM (return path)
sudo ip route add 10.10.10.0/24 via 10.10.20.1 dev ens33

$ tracepath -n -m 3 10.10.20.50       (on Kali)
 1:  10.10.10.1      1.755ms
 2:  10.10.20.50     2.287ms reached
```

Session from Kali through the firewall (excerpt)

```
$ ssh root@10.10.20.50
root@10.10.20.50's password:
root@svr04:~# whoami
root
root@svr04:~# uname -a
Linux svr04 6.1.0-21-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.1.90-1 (2024-05-03)
x86_64 GNU/Linux
root@svr04:~# id
uid=0(root) gid=0(root) groups=0(root)
root@svr04:~# ls -la /root
root@svr04:~# exit
```

An earlier repeat session was still logged as 10.10.20.5: the pinned route on Kali had reverted to the NAT path (tracepath showed 192.168.75.2 as hop 1), so the route has to be re-applied after a network restart. After it was re-applied, the session above was logged with 10.10.10.10.



Figure 5.6 — Kali during a repeat session to the honeypot (14:07 on Kali's clock). Cowrie logged these commands with source 10.10.20.5, because the pinned route had reverted to the NAT path.



Figure 5.7 — Cowrie's log after the fix: every command is recorded with source 10.10.10.10, including the typo.

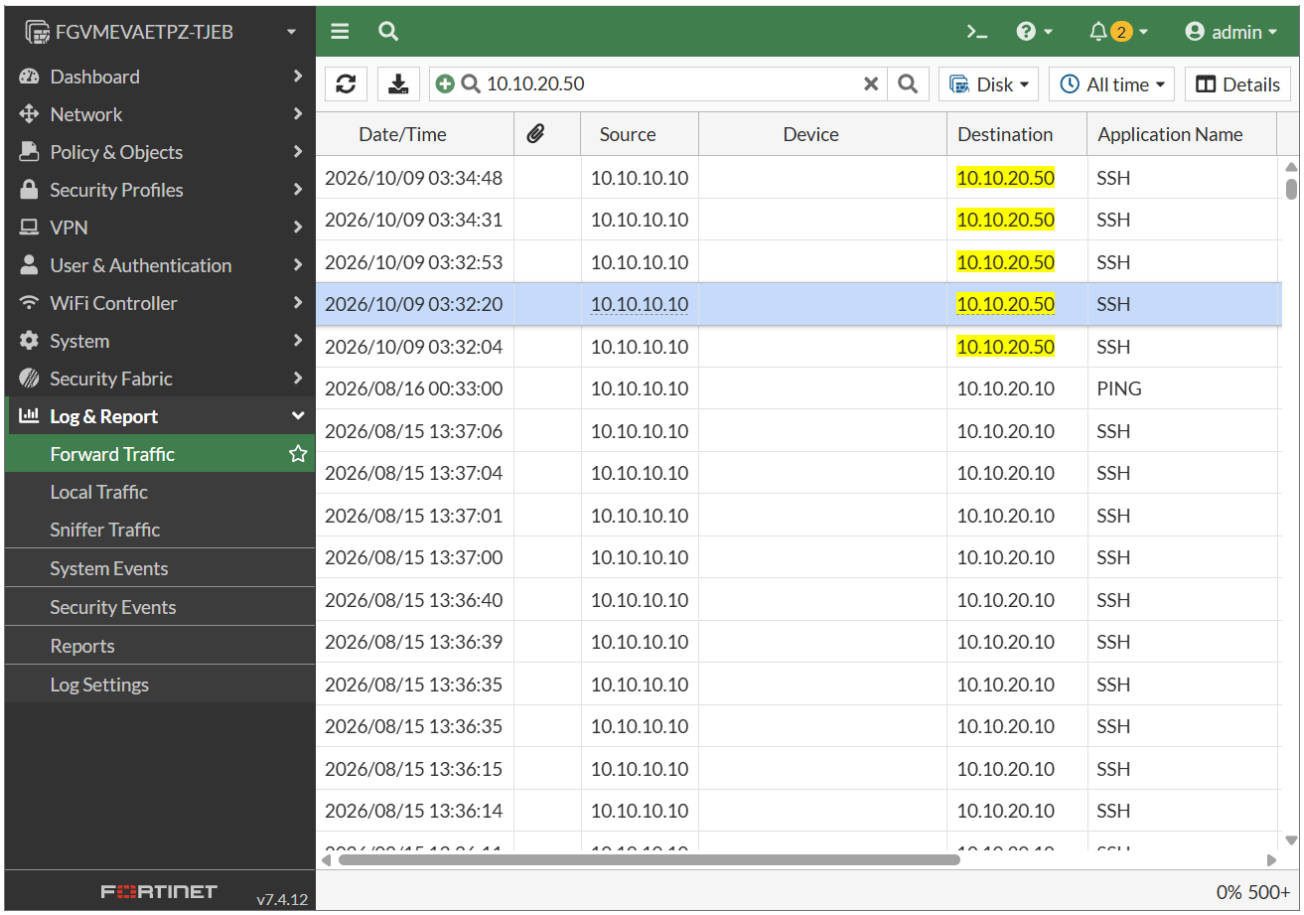


Figure 5.8 — FortiGate Forward Traffic filtered on the honeypot address: sessions from 10.10.10.10 are logged with the source unchanged (NAT is disabled).

Getting Cowrie events into Splunk

The first ingestion arrived in Splunk as one merged event stamped with its arrival time, because the `cowrie` sourcetype had no line-breaking or timestamp rules. A stanza on the Splunk server fixed this, after which each command is its own event with Cowrie's own timestamp.

props.conf on the Splunk server

```
[cowrie]
SHOULD_LINEMERGE = false
LINE_BREAKER = ([\r\n]+)
KV_MODE = json
TIME_PREFIX = "timestamp":"
TIME_FORMAT = %Y-%m-%dT%H:%M:%S.%6NZ
MAX_TIMESTAMP_LOOKAHEAD = 40
TZ = UTC
```

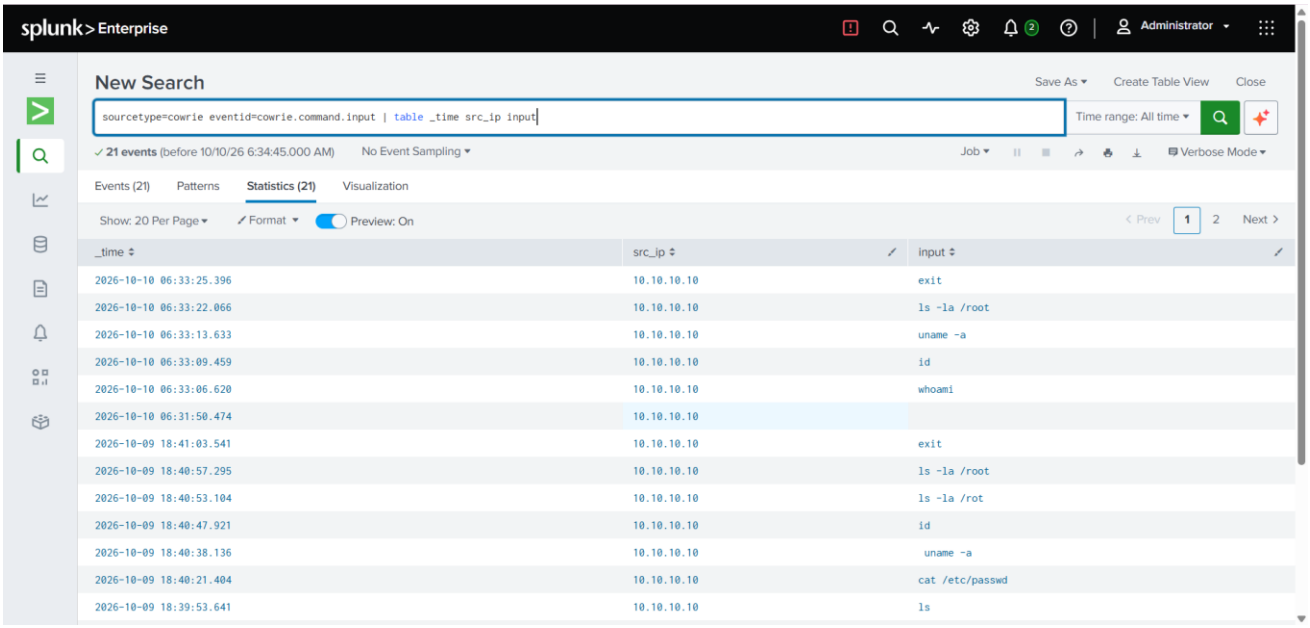


Figure 5.9 — Cowrie commands in Splunk, one row each with real timestamps and source 10.10.10.10.

Limitations. The `ip route` entries are not persistent and must be reapplied after a reboot, and Cowrie does not start by itself after a VM restart.

6. Security Orchestration & Automated Response (Shuffle SOAR)

Shuffle is self-hosted. The playbook receives a source IP on a webhook, checks FortiGate through the REST API for an existing address object, and creates one only if it is missing. Building it required recovering the platform's backend database from a memory-exhaustion crash loop and discovering that Shuffle's Filter action is the wrong tool for single-value conditions; connection-line conditions are used instead.

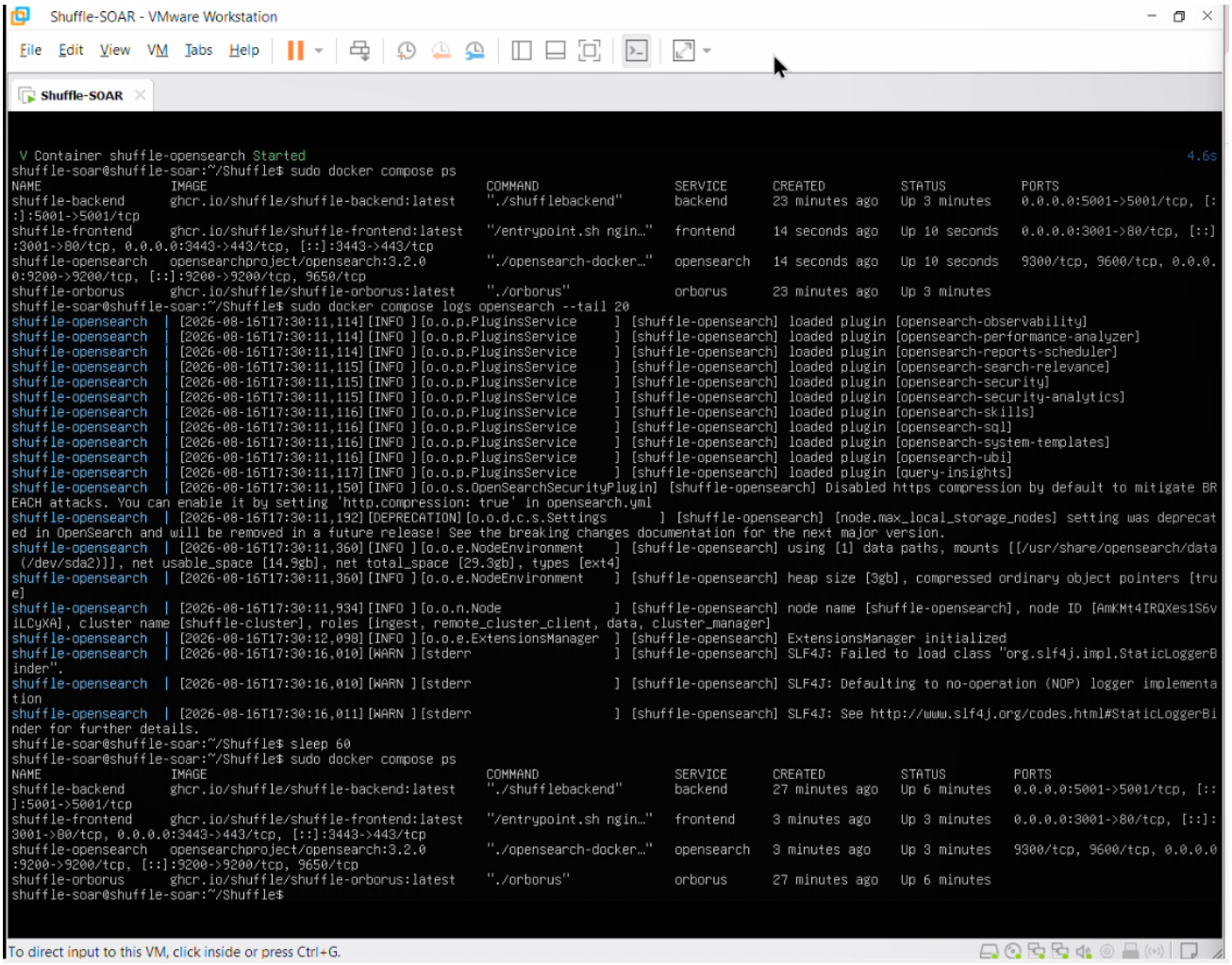


Figure 6.1 — Shuffle's backend (OpenSearch) stable after the out-of-memory crash loop was resolved.

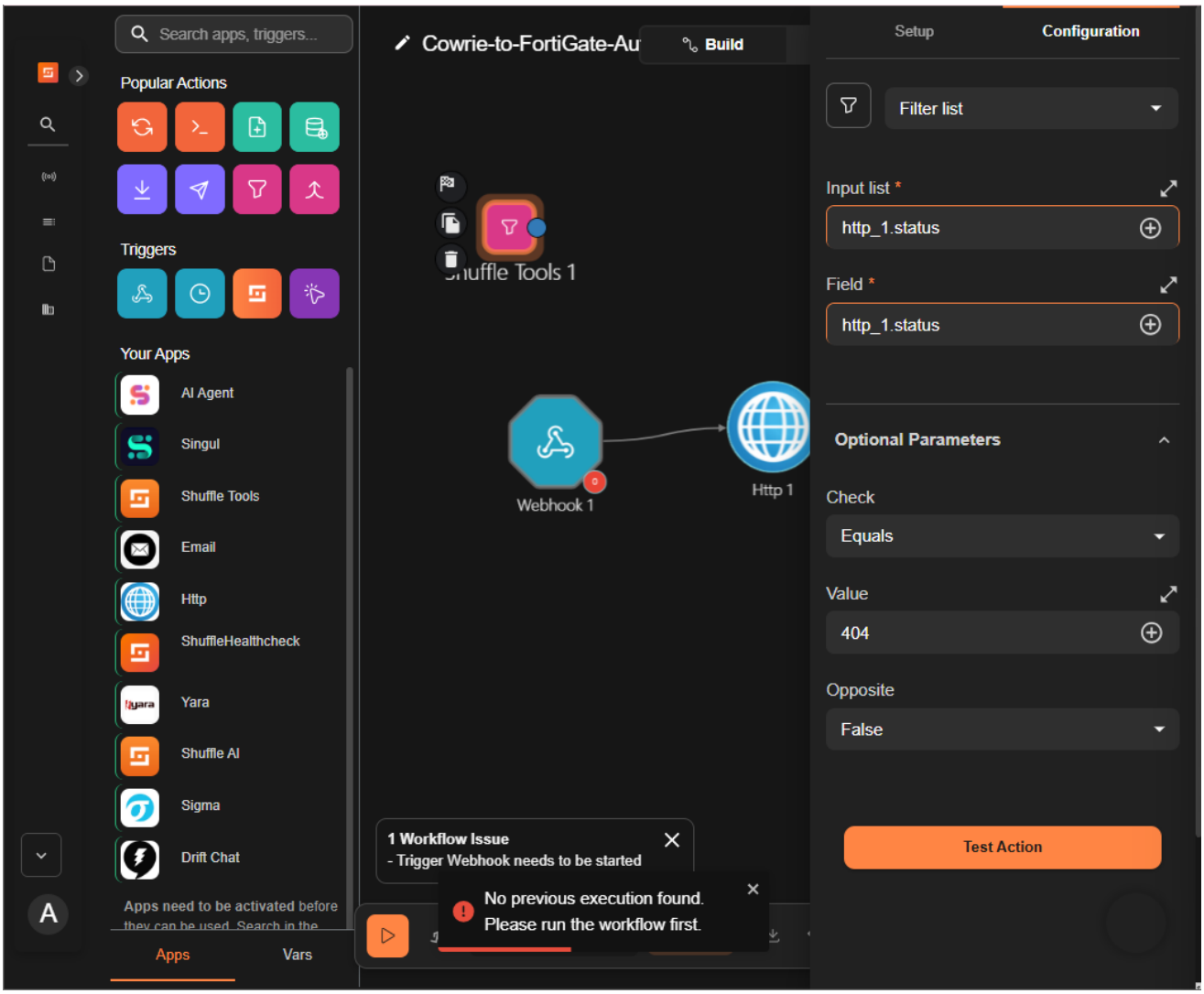


Figure 6.2 — First conditional-logic attempt with the Filter action, before it was identified as the wrong tool.

Step	Action
1	Webhook trigger receives `{"src_ip": ...}`
2	HTTP GET to the FortiGate address API for `cowrie-block-<ip>`
3	Condition on the connection line: continue only if the GET returned 404 (object absent)
4	HTTP POST creates the address object (`<ip>/32`)

First run: the object is created

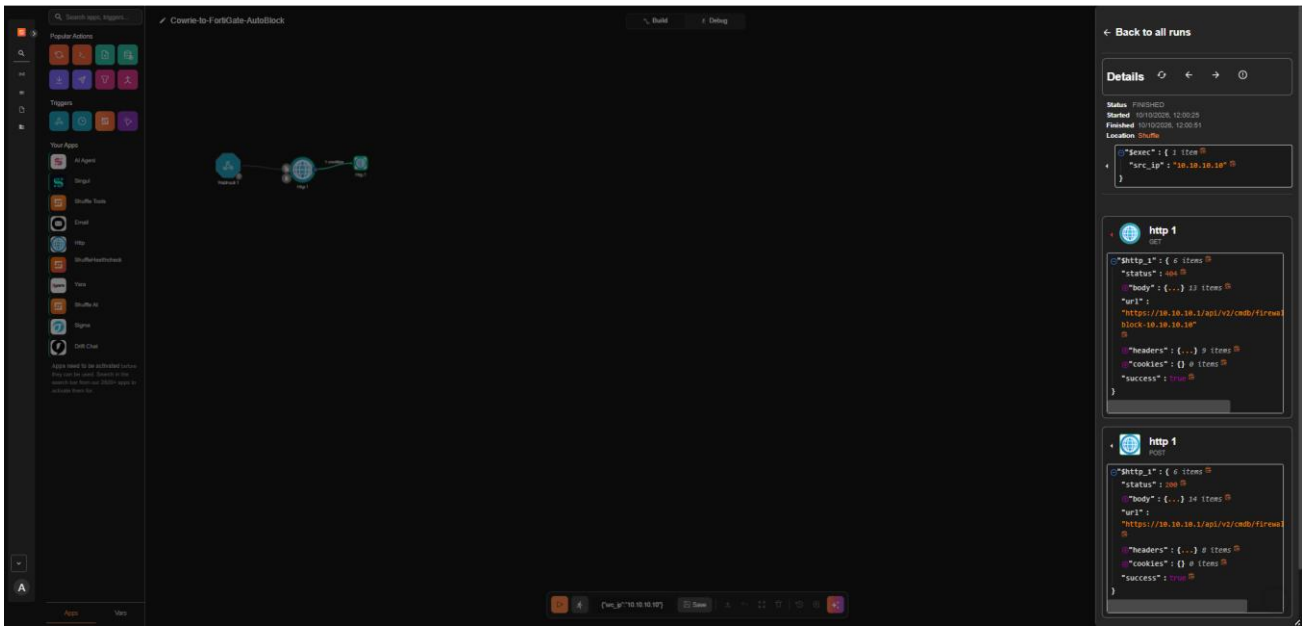


Figure 6.3 — Run with src_ip 10.10.10.10: the GET returns 404 (not found) and the POST returns 200 (created).

Finding: created is not enforced

The new object had a reference count of 0, and the only policy in the firewall was the accept-all Attacker-to-Victim rule. An address object on its own blocks nothing.

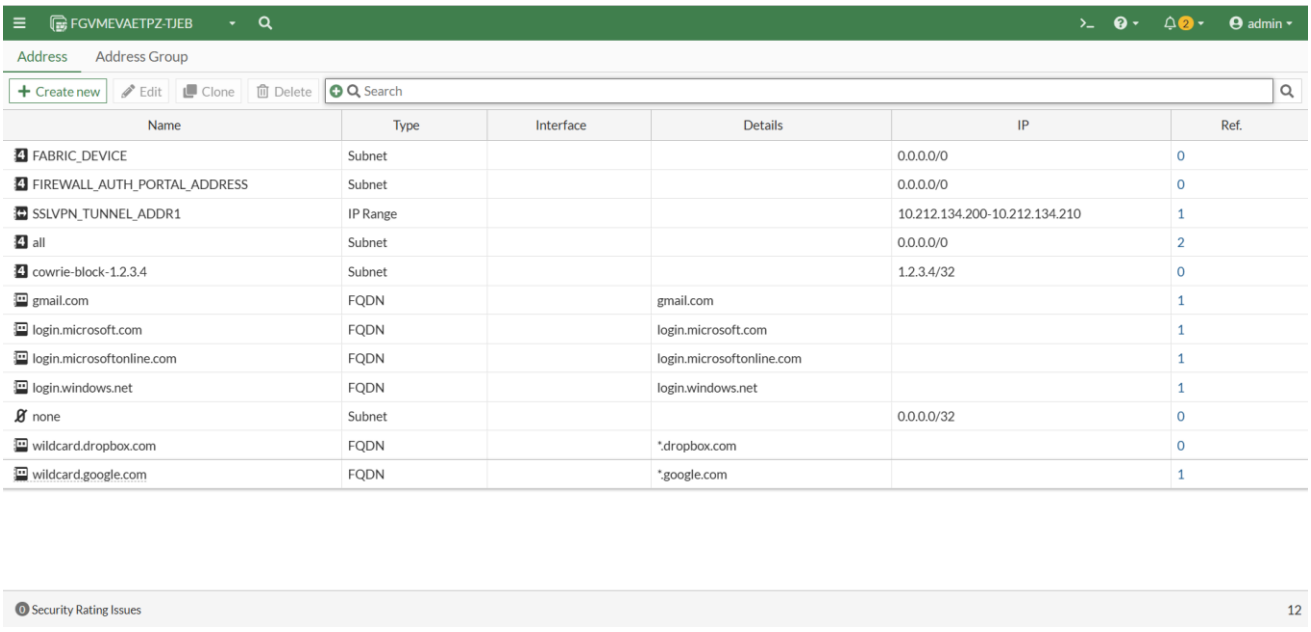


Figure 6.4 — The address object created by the playbook; the reference count (Ref.) is 0, so no policy uses it.

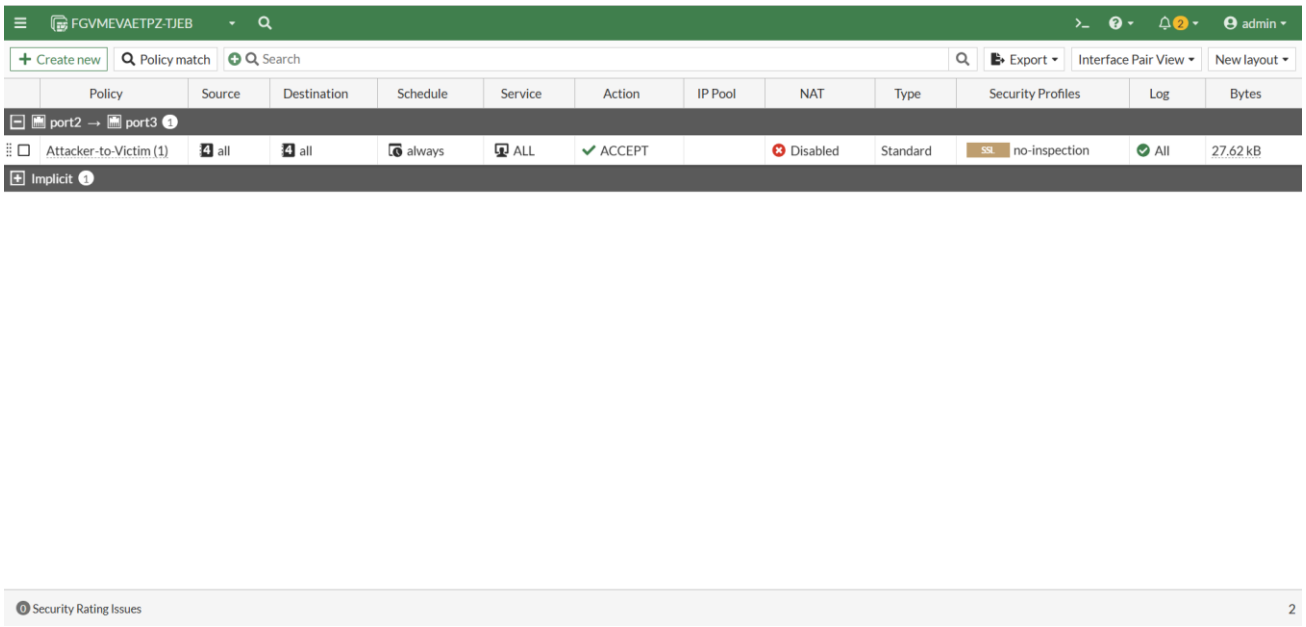


Figure 6.5 — Before: a single accept-all policy.

Enforcing the block

A deny policy was placed above the accept policy, since FortiGate evaluates policies from the top. For the test its source was the individual object.

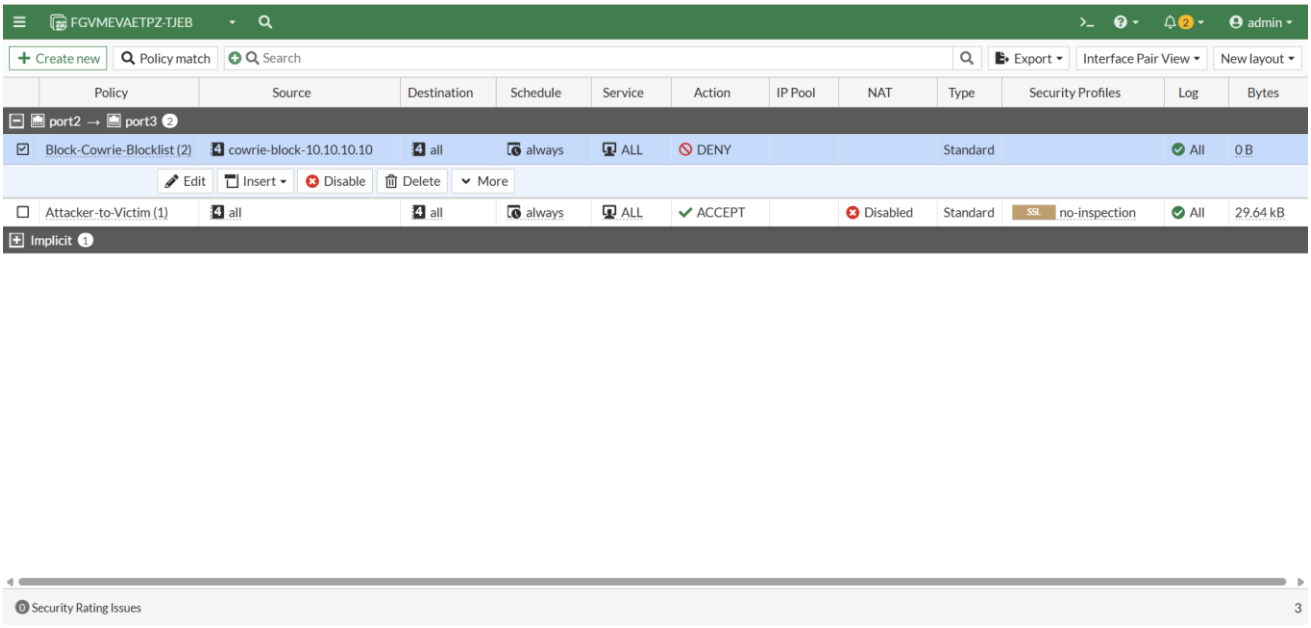


Figure 6.6 — After: Block-Cowrie-Blocklist (deny) above Attacker-to-Victim (accept).

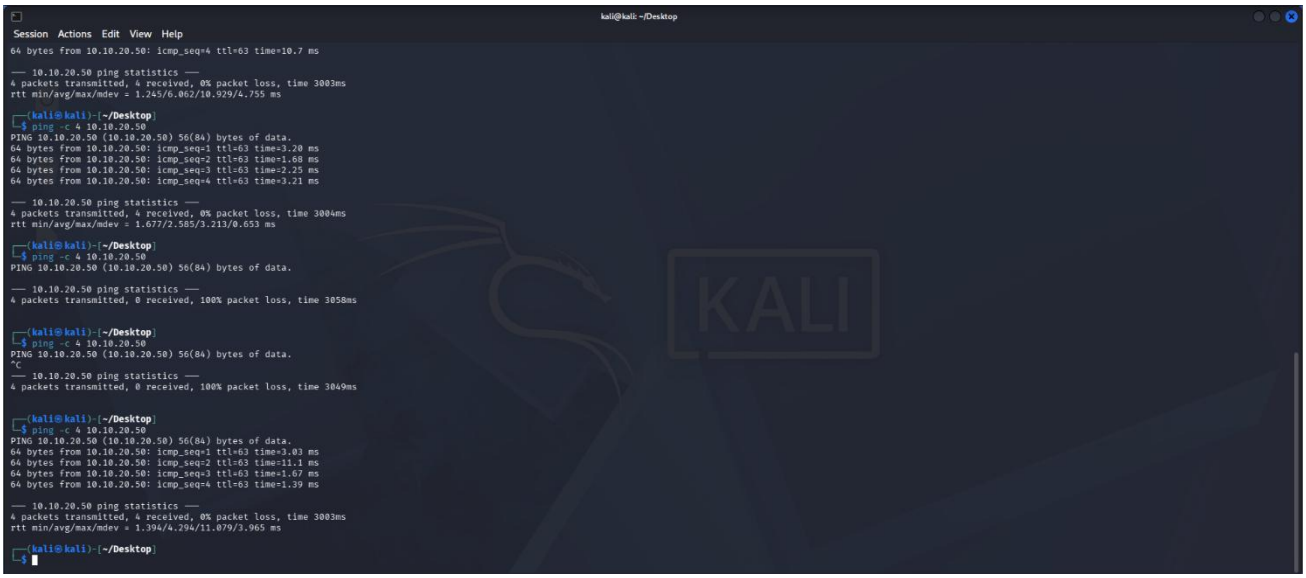
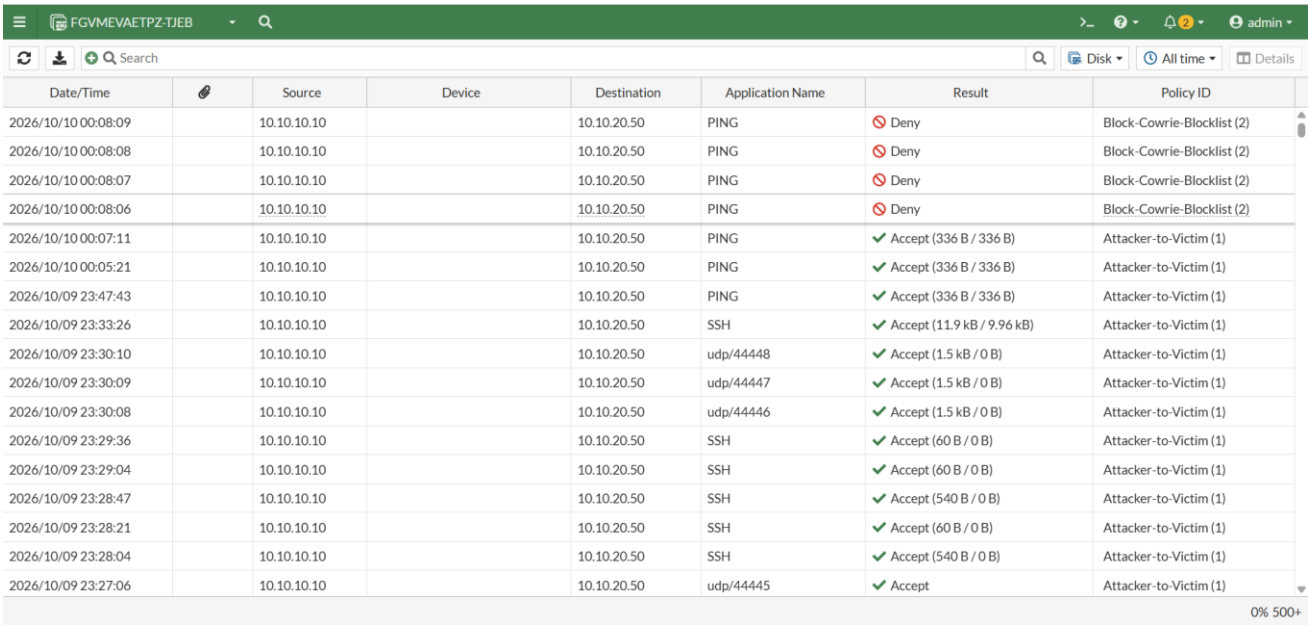


Figure 6.7 — Pings from Kali to the honeypot: replies before the block, 100% packet loss while the deny policy matched, and replies again after Kali's object was removed from the group.



Date/Time	Source	Device	Destination	Application Name	Result	Policy ID
2026/10/10 00:08:09	10.10.10.10		10.10.20.50	PING	Deny	Block-Cowrie-Blocklist (2)
2026/10/10 00:08:08	10.10.10.10		10.10.20.50	PING	Deny	Block-Cowrie-Blocklist (2)
2026/10/10 00:08:07	10.10.10.10		10.10.20.50	PING	Deny	Block-Cowrie-Blocklist (2)
2026/10/10 00:08:06	10.10.10.10		10.10.20.50	PING	Deny	Block-Cowrie-Blocklist (2)
2026/10/10 00:07:11	10.10.10.10		10.10.20.50	PING	Accept (336 B / 336 B)	Attacker-to-Victim (1)
2026/10/10 00:05:21	10.10.10.10		10.10.20.50	PING	Accept (336 B / 336 B)	Attacker-to-Victim (1)
2026/10/09 23:47:43	10.10.10.10		10.10.20.50	PING	Accept (336 B / 336 B)	Attacker-to-Victim (1)
2026/10/09 23:33:26	10.10.10.10		10.10.20.50	SSH	Accept (11.9 kB / 9.96 kB)	Attacker-to-Victim (1)
2026/10/09 23:30:10	10.10.10.10		10.10.20.50	udp/44448	Accept (1.5 kB / 0 B)	Attacker-to-Victim (1)
2026/10/09 23:30:09	10.10.10.10		10.10.20.50	udp/44447	Accept (1.5 kB / 0 B)	Attacker-to-Victim (1)
2026/10/09 23:30:08	10.10.10.10		10.10.20.50	udp/44446	Accept (1.5 kB / 0 B)	Attacker-to-Victim (1)
2026/10/09 23:29:36	10.10.10.10		10.10.20.50	SSH	Accept (60 B / 0 B)	Attacker-to-Victim (1)
2026/10/09 23:29:04	10.10.10.10		10.10.20.50	SSH	Accept (60 B / 0 B)	Attacker-to-Victim (1)
2026/10/09 23:28:47	10.10.10.10		10.10.20.50	SSH	Accept (540 B / 0 B)	Attacker-to-Victim (1)
2026/10/09 23:28:21	10.10.10.10		10.10.20.50	SSH	Accept (60 B / 0 B)	Attacker-to-Victim (1)
2026/10/09 23:28:04	10.10.10.10		10.10.20.50	SSH	Accept (540 B / 0 B)	Attacker-to-Victim (1)
2026/10/09 23:27:06	10.10.10.10		10.10.20.50	udp/44445	Accept	Attacker-to-Victim (1)

0% 500+

Figure 6.8 — FortiGate Forward Traffic: four denied pings from 10.10.10.10 under Block-Cowrie-Blocklist, right after the accepted ones.

Second run: no duplicate

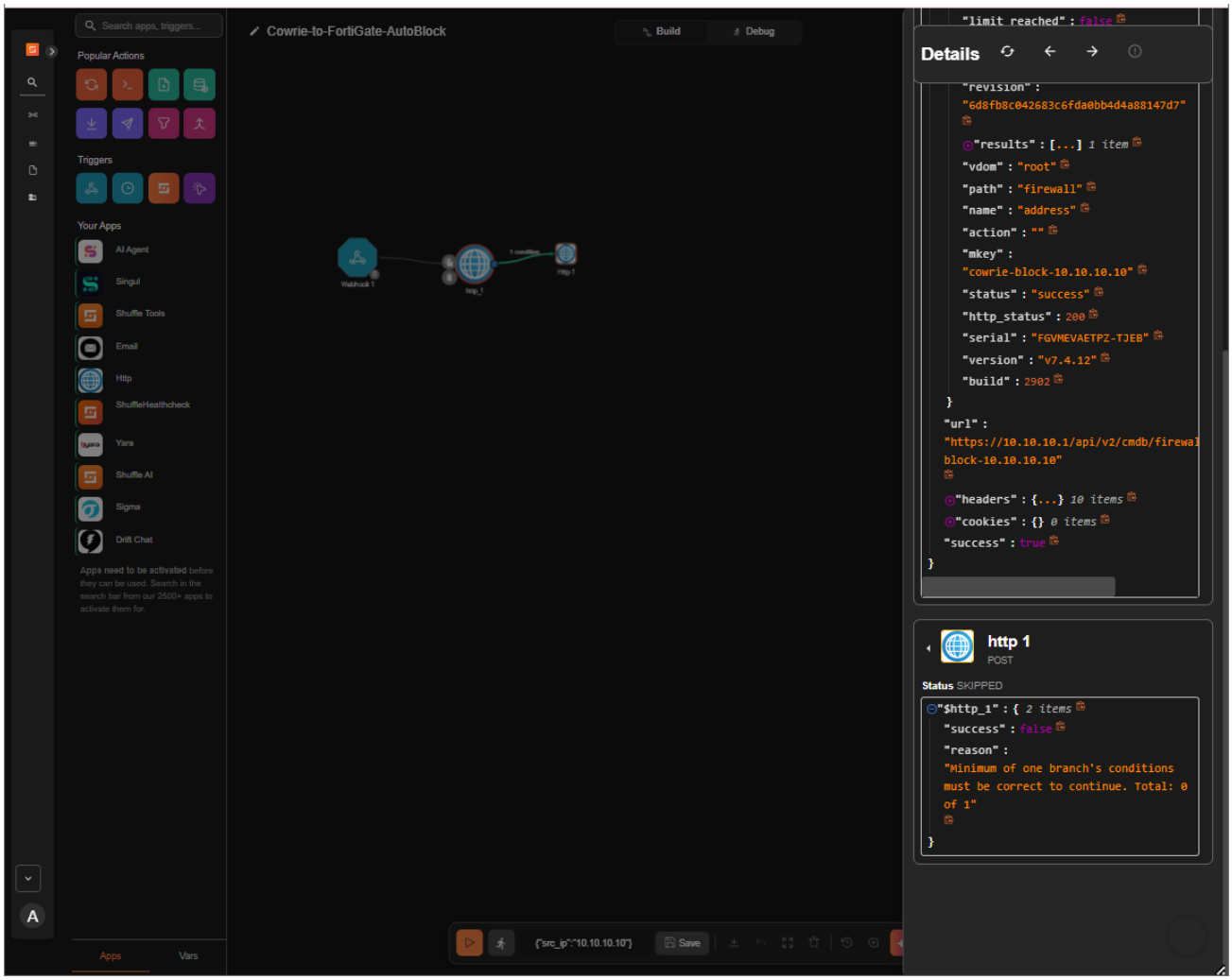


Figure 6.9 — Same input again: the GET returns 200 (object exists) and the POST is skipped because the condition is not met.

Skipping creation shows the playbook is idempotent: re-running it for an already blocked address neither errors nor creates duplicates.

Final design

Afterwards the policy source was changed to an address group, `cowrie-blocklist`, so further blocked addresses can be added to the group without editing the policy. Kali's object was then removed from the group so lab traffic could resume.

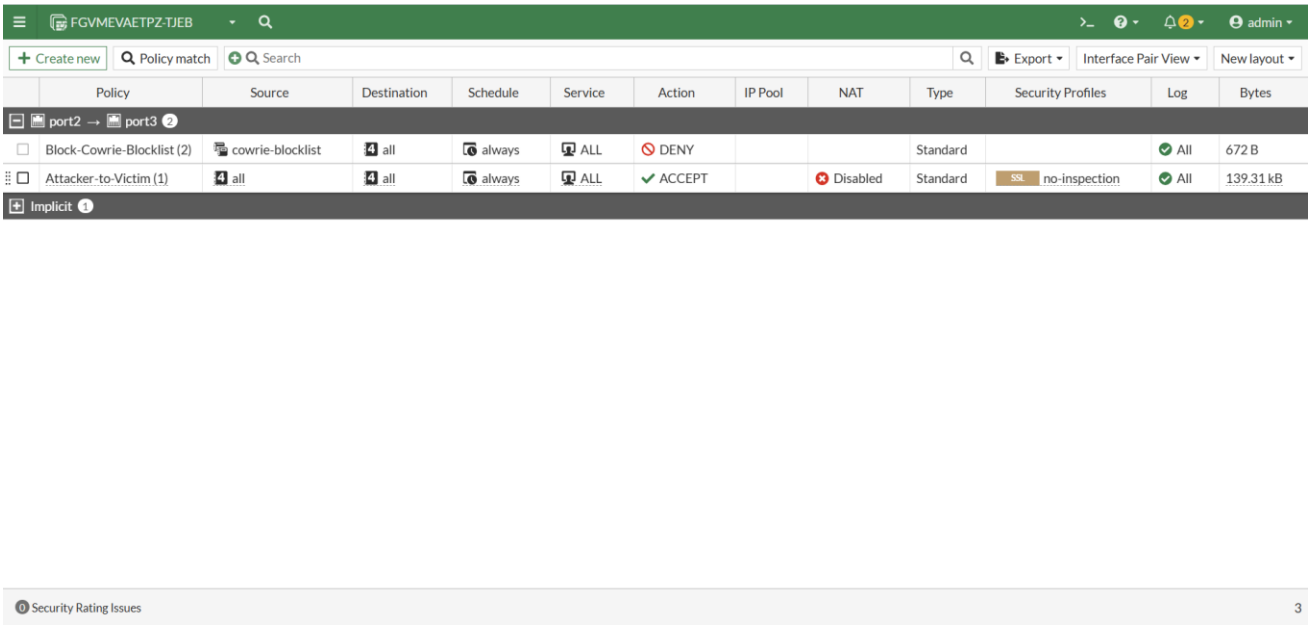


Figure 6.10 — Final policy: Block-Cowrie-Blocklist uses the cowrie-blocklist group.

Limits. The playbook only creates the address object; placing it in the group is manual and would need a third workflow step. The run was started manually with a test payload. The API account is restricted to one trusted host but uses a broad admin profile.

7. Detection Latency Measurement

Latency was calculated from Suricata's own timestamps: the alert time minus the start time of the flow, per signature. This is sensor alert latency, not end-to-end Mean Time to Detect, which would also include analyst response. Threshold rules cannot alert before their count is reached, so the figures partly reflect rule design.

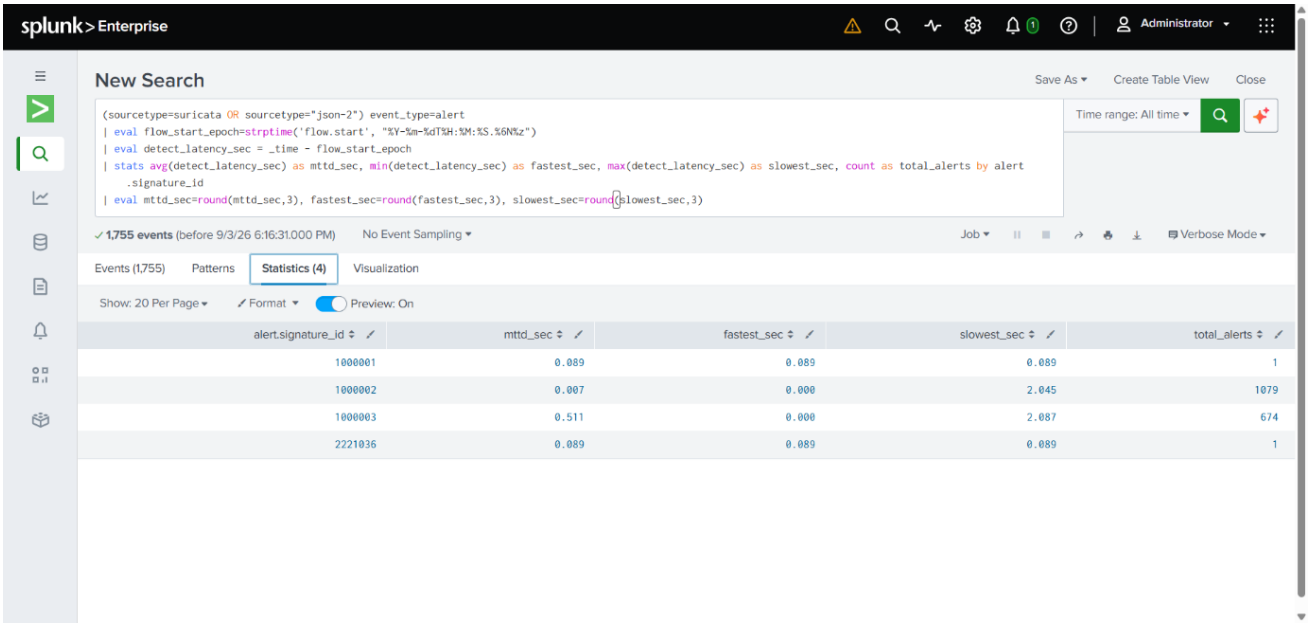


Figure 7.1 — Per-signature alert latency in Splunk (revision 1 rules).

SID	Rule	Average	Fastest	Slowest	Alerts
1000001	SQL injection (UNION SELECT)	0.089 s	0.089 s	0.089 s	1
1000002	SSH connection burst	0.007 s	0.000 s	2.045 s	1,079
1000003	SYN scan	0.511 s	0.000 s	2.087 s	674
2221036	Stock rule (not a custom rule)	0.089 s	0.089 s	0.089 s	1

The 1,755 total alerts are the 1,754 from custom rules plus one from a stock rule. The SQL injection figure is a single observation, not a statistical average. These numbers were measured with the revision 1 rules; revision 2 alerts once per interval, so its alert counts are much lower.

8. ATT&CK Coverage Summary

Technique	Rule (SID)	Test used	Atomic Red Team	Result
T1190 Exploit Public-Facing Application	SQLi in URI / body (1000001, 1000004)	Manual `curl` payloads, including URL-encoded	No test exists	Detected
T1110.001 Password Guessing	SSH connection burst (1000002)	Hydra (rev 1); connection burst (rev 2)	Tests exist, none is network SSH	Detected at connection level
T1046 Network Service Discovery	High-rate SYN scan (1000003)	Atomic T1046 and Nmap	Yes	Detected

9. Limitations and Next Steps

Limitations

- The Shuffle playbook was triggered manually with a test payload, not by a live Cowrie event, and adding the new object to the deny policy's address group is manual.
- The SQL injection rules are keyword-based on decoded HTTP buffers; heavier obfuscation and non-HTTP transports can still evade them.
- The scan and SSH rules count TCP SYN packets, not distinct ports or authentication attempts, and were checked against only a small amount of benign traffic.
- The correlation search joins on address, not time, and the counts are cumulative over all testing.
- Detection latency is sensor alert latency measured with revision 1 rules, and one rule has a single sample.
- Routes added at the command line are not persistent, and Cowrie does not start automatically after a reboot.
- The FortiGate runs on an evaluation license with no signature updates and no inspection profile, and it allows HTTPS and SSH administration on the lab interfaces.
- The REST API account is limited to one trusted host but uses a broad admin profile.

Next steps

- Trigger the playbook from a live source (a Splunk alert on Cowrie events calling the webhook).
- Add a third workflow step that places the new object into the address group.
- Create a scoped admin profile for the API account and disable administrative access on the lab interfaces.
- Validate the distinct-port scan search in the repository and tune thresholds on realistic traffic.
- Make routes and the Cowrie service persistent across reboots.

10. Repository

Build notes, rules and searches: https://github.com/shiza2-p/soc-lab-detection-engineering

- `docs/`: day-by-day build and debugging notes, including the routing finding and the auto-block test (Day 12).
- `suricata-rules/`: the revision 2 rules.
- `splunk-searches/`: latency, hunting, correlation and distinct-port searches.
- `atomic-tests/`: Atomic Red Team test reference.

Credentials in the repository are shown as placeholders.